SIGA-INEBI

Manual técnico

Cómo está construido, validado y mantenido el sistema de gestión del Instituto Nacional de Educación Básica de Salcajá

Ficha del documento

Sistema	SIGA-INEBI — plataforma web académica, administrativa y operativa
Audiencia	Desarrolladores, revisores técnicos, responsables de despliegue y evaluadores
Base documental	Arquitectura, decisiones, desarrollo, requisitos, Docker, CI y pruebas
Estado del código analizado	Rama develop, commit 0996960
Versión del documento	1.0
Fecha de elaboración	29 de septiembre de 2026
Fecha de edición de presentación	27 de octubre de 2026
Estado	Edición de presentación y control documental

Documento técnico de referencia para mantenimiento, revisión y evolución del sistema.
Contenido técnico original preservado; presentación normalizada para entrega documental.

Cómo leer este manual

El manual sigue el orden pedido para el entregable: alcance y arquitectura, tecnologías y decisiones, backend y API, frontend, seguridad y auditoría, base de datos y calidad, y operación técnica. Se añade un capítulo final que consolida las brechas y las discrepancias entre documentación y código, y dos anexos con datos de referencia.

Convenciones

Marca	Significado
código	Nombre de archivo, comando, campo, endpoint o variable, tal como aparece en el repositorio.
No implementado	La capacidad se menciona en documentos o requisitos pero no existe en el código. Se indica su referencia.
Recuadro azul	Nota informativa.
Recuadro ámbar	Atención: comportamiento que puede sorprender o discrepancia documental.
Recuadro rojo	Capacidad no implementada o riesgo abierto.
[lee] · [escribe] · [destruye]	En los comandos: efecto sobre datos y archivos.

El sistema de un vistazo

Implementado y verificado en el código
Monolito modular Django + DRF con 12 apps y 69 modelos
API REST bajo /api/v1/ con OpenAPI y sobre de error central
Sesión por cookie con CSRF, bloqueo de cuenta e inactividad por rol
RBAC con 39 permisos atómicos y alcance obligatorio, multi-rol aditivo
Auditoría de escrituras, lecturas sensibles y denegaciones (ORM inmutable)
Frontend React con rutas protegidas y sin tokens en el navegador
CI con 7 jobs, cobertura mínima 70 % (backend) y 60 % (frontend)
Respaldo y recuperación separados para base y archivos con simulacro

Contenido

	Capítulo	Incluye
1	**Alcance y arquitectura**	Propósito · límites · monorepositorio · monolito modular · diagramas de componentes y despliegue · dominios y dependencias
2	**Tecnologías y decisiones**	Tecnologías · mapa de ADR · ADR por tema · restricciones · decisiones pendientes
3	**Backend y API**	Apps Django · capas · errores y respuestas · convenciones · versionado · autenticación · tareas · inconsistencias
4	**Frontend**	Estructura · rutas · protección de pantallas · sesión por cookies · consumo de API · datos en el navegador · flujos por dominio
5	**Seguridad y auditoría**	Principios · RBAC y alcances · cuentas y personas · sesión · cookies, CSRF y TLS · auditoría · minimización · secretos · archivos · brechas
6	**Base de datos y calidad**	Modelos conceptual, lógico (ER) y físico · diccionario y scripts · migraciones · pruebas · CI · ramas y PR · Definition of Done · trazabilidad
7	**Operación técnica**	Versiones y puertos · instalación · comandos · migraciones · Docker · respaldo y recuperación · monitoreo · despliegue · guías canónicas
8	**Brechas y discrepancias consolidadas**	Capacidades no implementadas · diferencias entre documentación y código
A	**Anexo A — Endpoints que consume el frontend**	142 endpoints con uso y parámetros
B	**Anexo B — Inventario de tablas**	69 tablas con campos, claves foráneas y restricciones

1. Alcance y arquitectura

SIGA-INEBI es la plataforma del *Instituto Nacional de Educación Básica de Salcajá* para concentrar la información académica, administrativa y operativa del establecimiento bajo **control de acceso**. Este capítulo describe para qué existe el sistema, qué queda fuera, cómo está organizado el repositorio y cómo se ejecuta hoy. Cada afirmación se contrasta con el **código**, *compose.yml*, los **ADR aceptados** y *docs/requirements/functional-scope.md*; donde un documento antiguo contradice el código, manda la **configuración actual** y la diferencia queda anotada.

1.1 Propósito

El sistema permite a personal autorizado del establecimiento:

- identificar personas institucionales y vincular cada cuenta a una persona;
- administrar ciclos, estructura académica, matrícula y movimientos del estudiante;
- capturar y gobernar la asistencia, incluida la credencial con QR opaco;
- registrar evaluación y emitir documentos a partir de plantillas;
- consultar alertas mínimas de inasistencia;
- dejar bitácora de operaciones y lecturas sensibles.

La operación prevista es la de una institución, en la zona horaria del establecimiento (**America/Guatemala**), durante la jornada lectiva. La interfaz debe poder usarse en escritorio y en teléfonos de gama baja (perfil de referencia: Android con 2 GB de RAM y viewport de 360 × 640, **docs/architecture/frontend-compatibility.md**). El escaneo QR exige cámara en un contexto seguro (HTTPS o el equivalente de desarrollo local).

1.2 Límites funcionales

El núcleo fundacional incluye identidad y autorización, persona y expediente, ciclo y estructura académica, matrícula, credencial y asistencia, documentos y justificaciones, evaluación mínima y auditoría. Esos límites están en **docs/requirements/functional-scope.md**.

Queda fuera de esta fase
Inventario y programa de alimentos (PD-009 pide confirmar si entra al roadmap)
Agenda y citas
Comunicación institucional completa
Tableros avanzados y reportería extensa
Importaciones masivas generales
Automatizaciones que no sostienen el núcleo

La meta de capacidad de referencia es un servidor de 1 vCPU y 2 GB con una matrícula de prueba de unos 500 estudiantes, hasta que el establecimiento confirme la cifra real (PD-001). Respaldo de base y de archivos son pilas distintas, con RPO de referencia de 24 h y RTO de referencia de 4 h (PD-002). El estudiante es sujeto de datos; los requerimientos actuales no lo confirman como usuario del sistema.

> **No implementado.** Roles definidos en los documentos como “Inventory Manager” y “Food Programme Manager” no tienen dominio implementado: no existen apps de inventario ni de alimentación en **backend/apps/**.

1.3 Monorepositorio

ADR-0001 fija un solo repositorio. Frontend, backend, documentación y automatización comparten historial para que un cambio de contrato, de pantalla y de requerimiento se revise junto. La separación de responsabilidades es la siguiente:

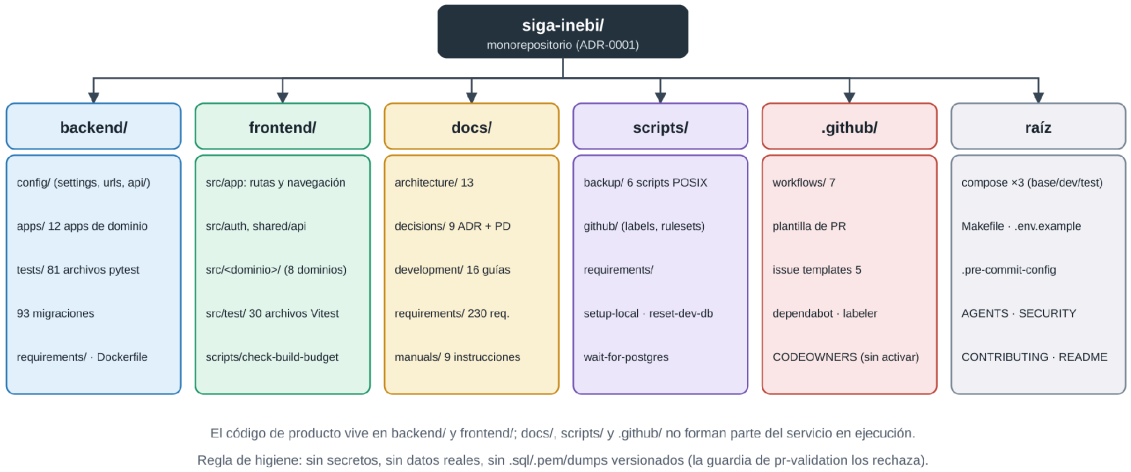


Figura 1. *Estructura del monorepositorio y contenido real de cada carpeta.*

Carpeta	Qué contiene	Qué no contiene
frontend/	Pantallas, rutas, sesión en el navegador y cliente HTTP (React 19, Vite 7, MUI 7, React Router 7, TanStack Query).	Reglas de negocio ni acceso directo a la base.
backend/	Apps por dominio, API REST, migraciones (93) y pruebas del servidor (81 archivos pytest).	Binarios de negocio dentro de las tablas.
docs/	Arquitectura (13 documentos), ADR, requerimientos, guías de desarrollo (16) y manuales.	Código ejecutable del producto.
scripts/	Respaldo y restauración (**backup/**), arranque local, reinicio de base de desarrollo y utilidades de GitHub y requerimientos.	Secretos y datos reales.
.github/	7 workflows, plantilla de PR, 5 plantillas de issue, Dependabot, etiquetado y **CODEOWNERS** (con marcadores por completar).	El runtime que atienden los usuarios.
Raíz	**compose.yml**, **compose.override.yml**, **compose.test.yml**, **Makefile**, **.env.example**, **.pre-commit-config.yaml**, **AGENTS.md**, **SECURITY.md**.	Archivos **.env** reales versionados.

scripts/backup/ respalda la base y los archivos por separado. **scripts/github/** configura etiquetas, rulesets y el tablero, y no forma parte del servicio escolar. **scripts/requirements/** ayuda a extraer y trazar requerimientos.

1.4 Monolito modular

ADR-0002 rechaza los microservicios al inicio. El backend es un solo proceso Django. Cada dominio es una app en **backend/apps/** con contrato interno propio, y ADR-0008 separa las capas dentro de cada app:

- **api/** recibe HTTP, llama consultas o servicios y serializa la respuesta;
- **queries.py** lee con el ORM y no conoce DRF ni el objeto **Request**;
- **services.py** escribe, aplica invariantes, transacciones y auditoría;
- **apps.common.exceptions** expresa errores de dominio sin depender del framework HTTP.

La API pública es REST con JSON bajo **/api/v1/**. La documentación OpenAPI vive en **/api/v1/docs/** y el esquema en **/api/v1/schema/**.

1.5 Diagrama de componentes

El diagrama corresponde al proceso que atiende una petición hoy: el navegador habla con React, React con la API, y la API con PostgreSQL, el disco de archivos y, fuera de la petición, con comandos programados por **cron**.

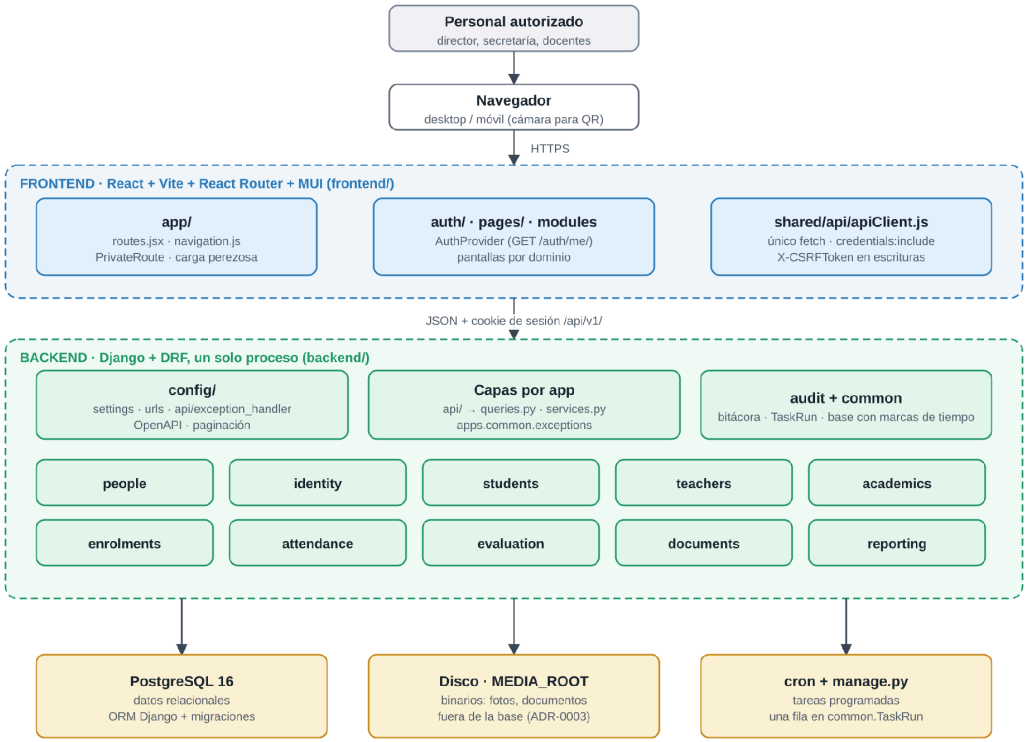


Figura 2. *Diagrama de componentes: frontend, backend modular y almacenamiento.*

Las apps instaladas en **backend/config/settings/base.py** son **common**, **people**, **identity**, **students**, **teachers**, **academics**, **enrolments**, **attendance**, **evaluation**, **reporting**, **documents** y **audit**. **common** no es un dominio de negocio: aporta el modelo con marcas de tiempo, el registro **TaskRun** y las excepciones compartidas. **evaluation** no monta su propio prefijo en **config/api/urls.py**: sus rutas se incluyen anidadas dentro de **academics/api/urls.py**.

1.6 Diagrama de despliegue

Hoy hay dos formas de ejecutar el sistema en una máquina de desarrollo y una forma de validarlo en GitHub. No existe un manifiesto de producción con varios nodos: la sección de producción del diagrama describe la configuración esperada, no algo orquestado en este repositorio.

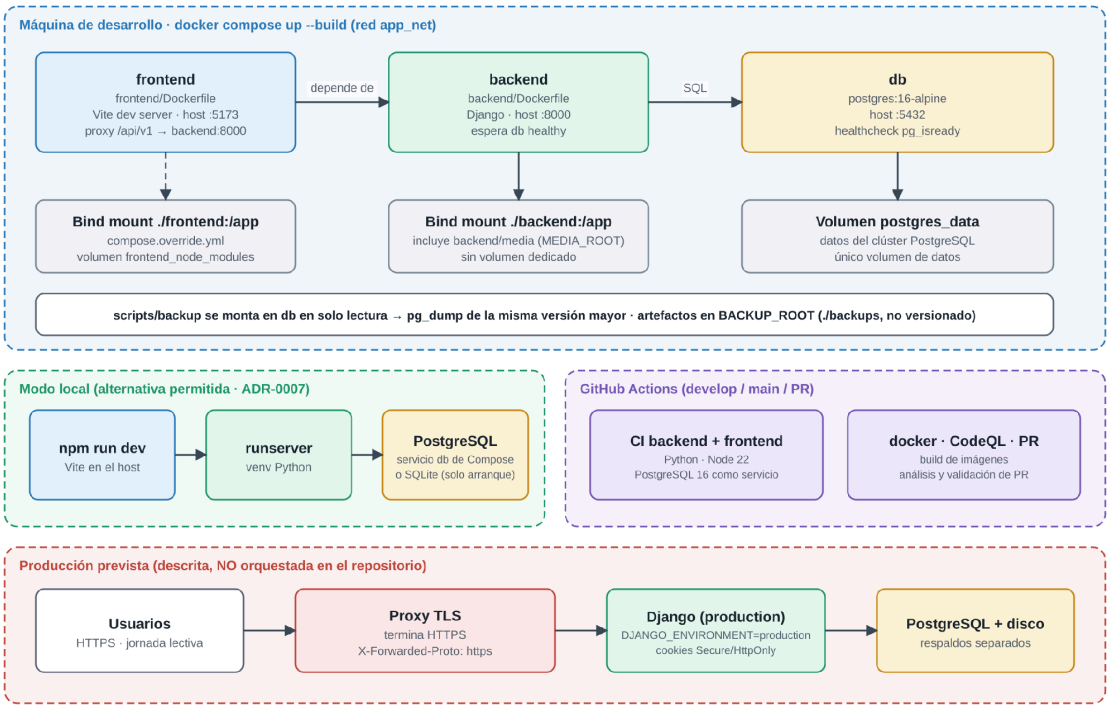


Figura 3. *Diagrama de despliegue: Compose, modo local, GitHub Actions y producción prevista.*

Compose

docker compose up --build levanta tres servicios en la red **app_net**:

Servicio	Imagen o build	Puerto en el host	Depende de	Comando de arranque
db	**postgres:16-alpine**	5432	—	PostgreSQL con **timezone** fijado a la zona del establecimiento
backend	**backend/Dockerfile** (**python:3.11-slim**)	8000	**db** sano	**migrate --noinput**, siembra opcional y **runserver 0.0.0.0:8000**
frontend	**frontend/Dockerfile** (**node:22-alpine**)	5173	**backend** sano	**npm run dev -- --host 0.0.0.0 --port 5173**

El único volumen de datos es **postgres_data**; el frontend usa además **frontend_node_modules**. **scripts/backup** se monta en el contenedor de PostgreSQL en solo lectura para que **pg_dump** sea de la misma versión mayor que el servidor. Los artefactos de respaldo van a **BACKUP_ROOT** (por defecto **./backups**), que no se versiona. En desarrollo, el frontend publica **VITE_API_URL=/api/v1** y el servidor de Vite reenvía **/api** y **/media** a **http://backend:8000**, de modo que el navegador no necesita conocer el nombre interno del contenedor.

compose.override.yml es el ajuste de desarrollo y se aplica solo: monta el código de **backend/** y **frontend/**, activa **DJANGO_DEBUG=true**, publica puertos y activa **SEED_DEMO_DATA_ON_START=true** (crea datos y un administrador demo). **compose.test.yml** aísla la base de pruebas con un PostgreSQL efímero en **tmpfs**.

> **No implementado.** Las imágenes actuales ejecutan el servidor de desarrollo de Django (**runserver**) y el servidor de desarrollo de Vite. No hay gunicorn/uwsgi ni una imagen de producción del frontend en el repositorio. Cualquier despliegue expuesto requiere trabajo adicional que hoy no está versionado.

Modo local

Se puede correr el frontend con npm y el backend con un entorno virtual, sin el stack completo (ADR-0007). La base puede ser el servicio **db** de Compose o SQLite; SQLite acelera un arranque local, pero las pruebas, el staging y la producción usan PostgreSQL y **require_postgresql()** lo exige.

Integración continua

GitHub Actions ejecuta en pull requests hacia **develop** y **main** (y en push a esas ramas) lint y pruebas del backend contra PostgreSQL 16, pruebas y compilación del frontend con Node 22, integración con Docker, análisis de seguridad y validación del pull request. El detalle está en la sección 6.7.

Producción prevista

La configuración de un ambiente expuesto está descrita, no orquestada en este repositorio. Debe usar **DJANGO_ENVIRONMENT=production** detrás de un proxy TLS que envíe **X-Forwarded-Proto: https**, con cookies de sesión **Secure**, **HttpOnly** y **SameSite=Lax**. Los secretos viven en el proveedor de despliegue, no en el repositorio. La disponibilidad comprometida es la jornada lectiva en días de clase (**docs/development/**`secure-deployment.md`).

1.7 Dominios y dependencias

El nombre de la app es el límite de código. El mapa funcional de **docs/architecture/domain-map.md** usa nombres más largos; aquí se usa el nombre que aparece en **INSTALLED_APPS**. La tabla resume responsabilidad y tamaño real de cada dominio.

App	Responsabilidad	Modelos	Migr.	Rutas API
people	Persona institucional: ancla de cuentas y actores	1	2	2
identity	Autenticación, cuentas, roles, permisos, alcances y sesión	6	13	14 (+9 en raíz)
students	Expediente, encargados, contactos, notas de salud y observaciones	6	7	15
teachers	Docente como actor institucional	1	2	3
academics	Institución, sedes, ciclos, niveles, grados, secciones, cursos, horarios y asignaciones docentes	18	17	49 (+7 de evaluación)
enrolments	Matrícula, reinscripción, retiros, traslados y movimientos	4	7	15
attendance	Credencial QR, captura, cierres, justificaciones y permisos	14	23	28

App	Responsabilidad	Modelos	Migr.	Rutas API
evaluation	Unidades, notas, recuperación y resultados congelados	6	4	7 (anidadas en academics)
documents	Tipos, plantillas, emisión, folios, verificación y descarga con token breve	9	15	19
reporting	Umbrales y alertas de inasistencia	2	1	4
audit	Bitácora de operaciones, lecturas sensibles y denegaciones	1	1	4
common	Base con marcas de tiempo, **TaskRun**, excepciones y monitoreo	1	1	3

Cada dominio depende de otros por dos vías distintas, que el manual anterior no separaba: las claves foráneas de sus modelos (dependencia de datos, que define el diagrama entidad-relación) y los **import** de su código de servicios y consultas (dependencia de comportamiento). La matriz siguiente se obtuvo analizando **backend/apps** y los modelos reales.



Figura 4. *Matriz de dependencias entre apps: claves foráneas frente a imports de código.*

Lo que la matriz muestra en la práctica:

- **people** no depende de ningún otro dominio por clave foránea; es la raíz del modelo (**UserAccount**, **Student**, **Guardian** y **Teacher** cuelgan de **Person**).
- **identity** depende de **students** y **academics** porque **ScopeGrant** puede acotar por estudiante, ciclo, grado, sección, materia y asignación; a su vez **students** referencia a **UserAccount** como autor de notas. Hay, por tanto, dependencia mutua entre ambos dominios, resuelta por referencias a cadena de texto de Django (**"app.Modelo"**).

- **academics** referencia **enrolments** (por los resultados congelados de promoción y de curso) y **enrolments** referencia **academics** (ciclo, grado y sección): existe otro par mutuo a nivel de datos.
- **audit** no es lectura pura: **AuditEvent.actor** es una clave foránea a **UserAccount** con **SET_NULL**, y conserva además **actor_label** para que el nombre sobreviva a una cuenta eliminada.
- **reporting** depende de **attendance** (las alertas nacen de alertas de asistencia) y de **identity** (quién las atiende).

Atención. La tabla de dependencias del manual anterior presentaba una jerarquía simple, con **people** en la base. Es cierto para **people**, pero el grafo real tiene pares mutuos (**identity** con **students**, y **academics** con **enrolments**) y una dependencia de código de **identity** hacia casi todas las apps. Refactorizarlo no es objeto de este manual; se documenta para que quien modifique alcances sepa qué apps toca.

Ciclo de vida de una petición

Toda petición de negocio recorre el mismo camino; los errores se resuelven en un solo punto.

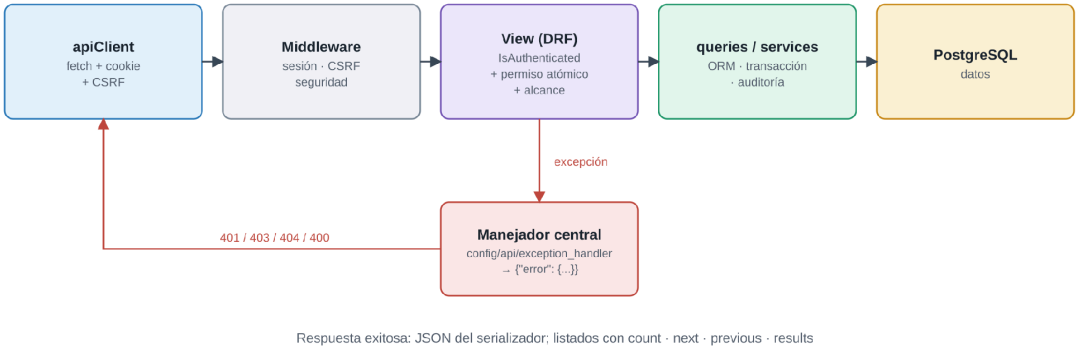


Figura 5. *Ciclo de vida de una petición y ruta de los errores hacia el manejador central.*

2. Tecnologías y decisiones

La selección tecnológica responde a los ADR vigentes y a la configuración verificable del repositorio. Donde no existe un ADR específico se indica expresamente, en lugar de atribuirle a un ADR una decisión que no toma.

2.1 Tecnologías principales

Tecnología	Versión real	Uso en SIGA-INEBI	Justificación y base de la decisión
React con Vite	React 19.1 · Vite 7 · React Router 7 · MUI 7	Interfaz web: rutas, pantallas por dominio y cliente HTTP único.	Cliente separado del backend para evolucionar la interfaz sin tocar reglas de negocio; Vite ofrece desarrollo ágil y un build con presupuesto de tamaño. **Sin ADR propio**: el stack lo fija **AGENTS.md** (“React/Vite en frontend”), **docs/architecture/system-context.md** y el contexto de ADR-0009.
Django REST Framework	Django 5.2 · DRF 3.17 · drf-spectacular 0.28	API JSON, validación, sesión y esquema OpenAPI.	ADR-0004 (API REST JSON) y ADR-0002 (monolito modular). Mantiene reglas de negocio y autorización en el backend; la interfaz no es la fuente de control de acceso.
PostgreSQL	16 (**postgres:16-alpine**) · psycopg 3.3	Base relacional en pruebas, staging y producción.	ADR-0003: SQLite solo en desarrollo local temprano; el comportamiento objetivo se valida contra PostgreSQL. Se usan restricciones específicas de PostgreSQL (**ExclusionConstraint**, índices únicos parciales).
Docker Compose	Python 3.11 · Node 22 · PostgreSQL 16	Ejecución reproducible del stack integrado y base de pruebas aislada.	ADR-0007: Compose como modo recomendado y ejecución local separada como alternativa permitida.
GitHub Actions	checkout v4 · setup-python v5 · setup-node v4	Integración continua, análisis de seguridad y validación de PR.	**Sin ADR propio.** Se justifica por la trazabilidad del monorepo (ADR-0001) y la exigencia de validar entornos reproducibles; la sección 6.7 describe lo que realmente se ejecuta.

2.2 Mapa de decisiones

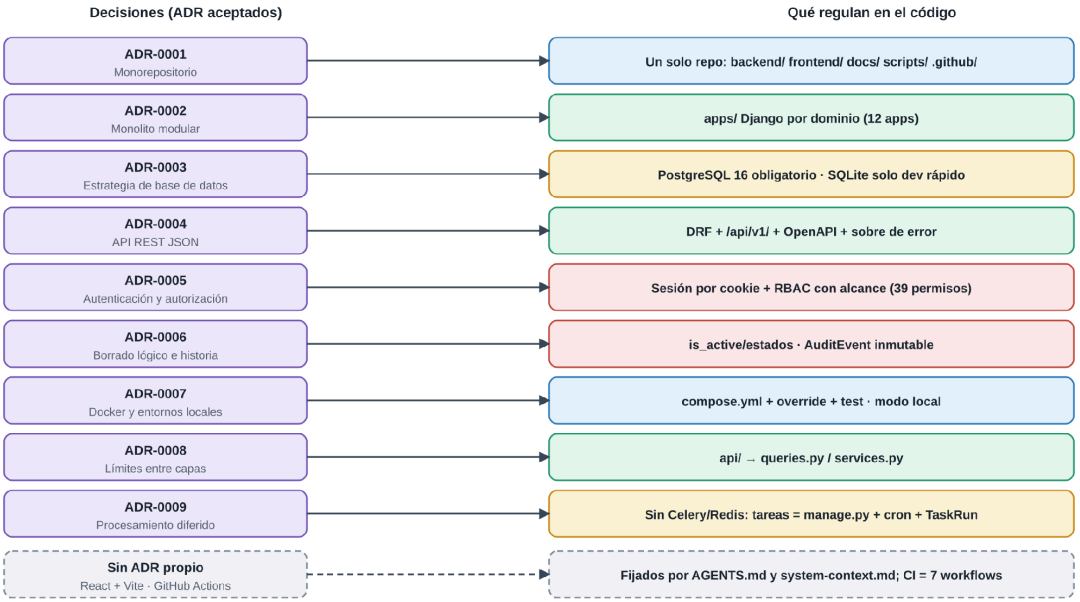


Figura 6. *Qué regula cada ADR aceptado en el código del repositorio.*

2.3 ADR que regulan cada tema

Las rutas son relativas a la raíz del repositorio.

Tema regulado	Decisión aplicada	ADR
Organización del código	Un solo repositorio con **frontend/**, **backend/**, **docs/**, **scripts/** y **.github/**.	**docs/decisions/ADR-0001-monorepository.md**
Arquitectura backend	Monolito modular: un proceso Django, una app por dominio, contratos internos claros.	**docs/decisions/ADR-0002-modular-monolith.md**
Base de datos	SQLite solo en desarrollo local temprano; PostgreSQL en pruebas automatizadas y ambientes no locales.	**docs/decisions/ADR-0003-database-strategy.md**
API	REST + JSON bajo **/api/v1/**, sobre de error uniforme y OpenAPI.	**docs/decisions/ADR-0004-rest-api.md**
Autenticación y autorización	Sesión web y RBAC con permisos atómicos y alcances (“scoped grants”), evaluados por operación.	**docs/decisions/ADR-0005-authentication-and-authorization.md**
Historia y borrado	Desactivación, revocación, versionado o estados antes que eliminación física en registros históricos.	**docs/decisions/ADR-0006-soft-delete-and-history.md**
Entornos	Docker Compose recomendado; ejecución local separada como alternativa.	**docs/decisions/ADR-0007-docker-and-local-environments.md**
Límites entre capas	Vistas/serializadores, **queries.py**, **services.py** y excepciones de aplicación.	**docs/decisions/ADR-0008-application-layer-boundaries.md**

Tema regulado	Decisión aplicada	ADR
Trabajos diferidos	Sin colas ni workers hasta contar con una decisión posterior que cubra persistencia, idempotencia, reintentos, observabilidad, auditoría, seguridad y pruebas.	**docs/decisions/ADR-0009-deferred-background-processing.md**

2.4 Restricciones vigentes

- Las reglas de negocio y las decisiones de autorización no pueden depender de la interfaz: el frontend puede ocultar opciones, pero el backend siempre valida.
- Las reglas de dominio se mantienen fuera de vistas y serializadores (ADR-0008); hay pruebas unitarias que lo vigilan.
- PostgreSQL es el motor objetivo; la compatibilidad con PostgreSQL debe preservarse en consultas, migraciones y pruebas. **config.settings.test** aborta si el motor no lo es.
- No se documentan Redis, Celery, workers, colas ni estados asíncronos como capacidades: los lotes (**grades/bulk/**, **re-enrolments/bulk/**, **official-issuance/batches/**) se procesan de forma síncrona en la petición.
- **AGENTS.md** exige leer requerimientos y ADR antes de modificar dominio, no implementar funcionalidades no solicitadas, no cambiar contratos públicos sin documentarlos y no marcar un RF como completado sin pruebas verificables.

3. Backend y API

SIGA-INEBI implementa su backend con Django 5.2 y Django REST Framework 3.17 bajo una arquitectura de monolito modular. Cada dominio funcional es una aplicación Django independiente; la API REST JSON es el contrato entre el frontend y la lógica de negocio. Se fundamenta en ADR-0002 (monolito modular), ADR-0004 (API REST JSON) y ADR-0008 (límites entre capas).

3.1 Estructura por aplicaciones Django

El backend vive en **backend/**. **config/** contiene la configuración de plataforma y **apps/** los doce dominios. Cada app conserva sus modelos, migraciones, consultas, servicios y capa de API.

```
backend/
├── config/
│   ├── settings/  base.py · development.py · staging.py · production.py · test.py
│   ├── urls.py    /admin/ y /api/v1/ → config.api.urls
│   └── api/       urls.py · views.py (health, auth) · exception_handler.py
├── apps/<dominio>/
│   ├── models.py · migrations/
│   ├── queries.py        lecturas con ORM
│   ├── services.py       escrituras, invariantes, auditoría
│   └── api/              urls.py · views.py · serializers.py
├── tests/  unit · api · integration · permissions · migrations
├── requirements/  (base y dev)
└── Dockerfile · docker-entrypoint.sh · manage.py · pyproject.toml
```

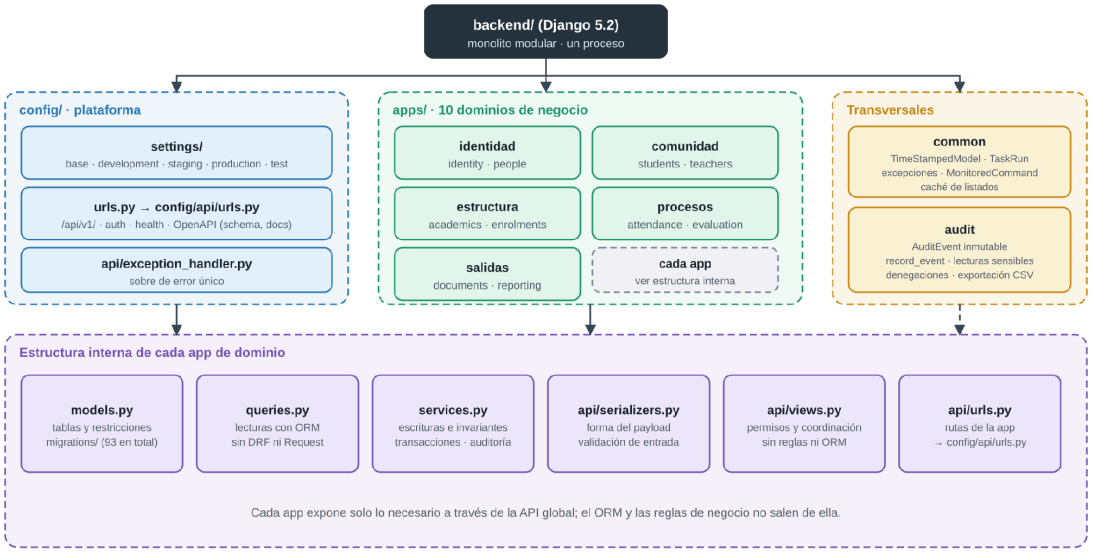


Figura 7. *Organización del backend: configuración, dominios, componentes transversales y estructura interna de cada app.*

Cada dominio registra sus rutas en **apps/<dominio>/api/urls.py**. **config/api/urls.py** las integra y se exponen bajo el prefijo global **/api/v1/**.

Prefijo (/api/v1/)	App	Rutas	Operaciones (aprox.)
(raíz) **health/**, **health/database/**, **schema/**, **docs/**, **auth/…**	**config.api**	9	~10
platform/	**common**	3	3
audit/	**audit**	4	4
identity/	**identity**	14	15
people/	**people**	2	6
students/	**students**	15	~31
teachers/	**teachers**	3	7
academics/ (incluye evaluación)	**academics** + **evaluation**	49 + 7	85 + 10
attendance/	**attendance**	28	31
reporting/	**reporting**	4	5
documents/	**documents**	19	25
enrolments/	**enrolments**	15	16

Nota. Los conteos salen de analizar **apps/*/api/urls.py** y las vistas; son aproximados (cuatro vistas **RetrieveUpdateDestroyAPIView** cuentan PUT y PATCH por separado). La fuente exacta es el esquema **GET /api/v1/schema/**. Total: ~163 rutas montadas por apps más 9 en la raíz.

3.2 Límites entre vistas, serializadores, consultas y servicios

La implementación mantiene una separación explícita entre transporte HTTP, validación de payloads, lecturas y reglas de negocio. Reduce el acoplamiento entre dominios y evita que las reglas críticas dependan de la interfaz web o de DRF.

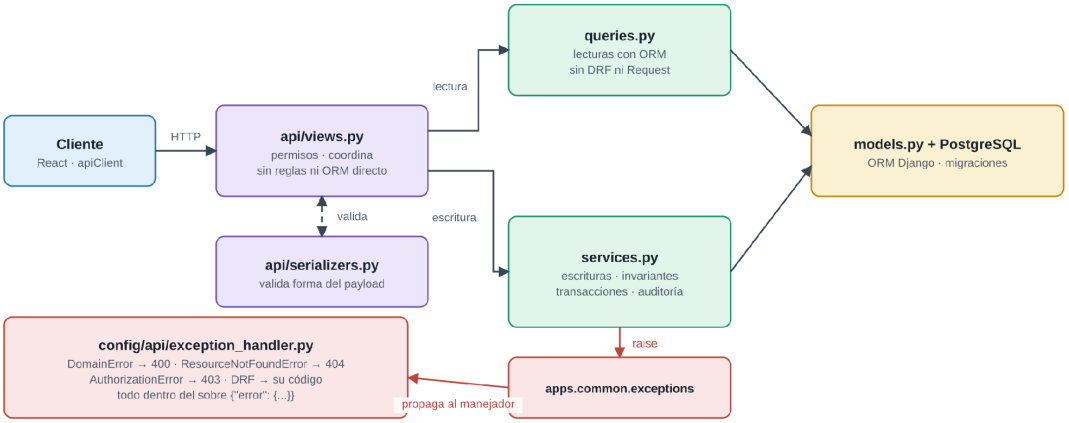


Figura 8. *Capas dentro de una app y camino de las excepciones.*

Capa	Ubicación	Responsabilidad	No debe contener
Vistas	**apps/<dominio>/api/views.py**	Reciben HTTP, aplican permisos, coordinan serializadores y delegan.	Reglas de negocio, transacciones complejas o consultas ORM directas.
Serializadores	**apps/<dominio>/api/serializers.py**	Validan estructura y tipos del payload; convierten entre JSON y objetos de aplicación.	Reglas de negocio, autorización contextual o persistencia compleja.
Consultas	**apps/<dominio>/queries.py**	Lecturas con ORM: filtros, ordenamientos, composición eficiente.	Dependencias de DRF, **Request**, respuestas HTTP o excepciones de transporte.
Servicios	**apps/<dominio>/services.py**	Casos de uso de escritura: invariantes, transacciones, auditoría.	Dependencia directa de vistas, serializadores o respuestas HTTP.
Excepciones de aplicación	**apps/common/exceptions.py**	**DomainError**, **ResourceNotFoundError**, **AuthorizationError**.	Serialización de respuestas o códigos HTTP.
Manejador de excepciones	**config/api/exception_handler.py**	Traduce errores de aplicación y de DRF a un sobre uniforme.	Reglas particulares de un dominio.

3.3 Manejo central de errores y formato de respuestas

REST_FRAMEWORK['EXCEPTION_HANDLER'] apunta a **config.api.exception_handler.api_exception_handler**. Las vistas no capturan errores de dominio para construir respuestas a mano: las excepciones se propagan al manejador, que produce un sobre **{"error": {"status_code", "detail"}}**.

Situación	Excepción o fuente	HTTP	Forma de error.detail
Regla de negocio incumplida	**DomainError**	400	Texto de la regla incumplida.

Situación	Excepción o fuente	HTTP	Forma de error.detail
Recurso inexistente	**ResourceNotFoundError**	404	Objeto **{"detail": "…"}** (igual que **NotFound** de DRF).
Permiso o alcance insuficiente	**AuthorizationError**	403	Objeto **{"detail": "…"}** (igual que **PermissionDenied**).
Validación, autenticación u otro error de DRF	Excepción estándar de DRF	Según corresponda	Estructura original de DRF (por campo, lista o **detail**).
Sesión expirada por inactividad	**SessionIdleTimeoutMiddleware**	401	**{"error": {"detail": "…"}}** sin **status_code**, con **X-SIGA-Session-Expired: 1** y **Cache-Control: no-store**.

Ejemplo de regla de negocio incumplida (400) y de recurso inexistente (404):

```
{ "error": { "status_code": 400,
             "detail": "La operación solicitada incumple una regla de negocio." } }

{ "error": { "status_code": 404,
             "detail": { "detail": "No se encontró el recurso solicitado." } } }
```

Atención. El sobre es uniforme, pero **detail** no tiene un tipo único (texto, objeto o estructura de DRF) y el 401 por inactividad no incluye **status_code**. Por eso **frontend/src/shared/api/apiClient.js** extrae el mensaje con una función tolerante a las tres formas. Además, algunos mensajes de dominio están en inglés (p. ej. “Actor lacks the required permission.”) aunque la convención pide mensajes funcionales en español.

3.4 Convenciones de API

- Estilo REST con JSON; recursos en plural; nombres técnicos de recursos y campos en inglés; mensajes funcionales y documentación visible en español.
- Fechas y horas en ISO 8601; **USE_TZ = True**, zona del establecimiento **America/Guatemala** y **LANGUAGE_CODE = "es-gt"**.
- Paginación estándar **PageNumberPagination** (parámetro **page**, **PAGE_SIZE = 25**) con **count**, **next**, **previous** y **results**.
- Identificadores opacos (**public_id** UUID) cuando conviene proteger detalles internos.
- Sin borrado físico en recursos históricos: **DELETE** significa desactivación lógica (ADR-0006).
- Toda la API exige sesión, salvo endpoints públicos declarados: **health/**, **health/database/**, **auth/login/**, **auth/logout/**, **auth/me/**, **accounts/activate/**, **password-resets/consume/** y **documents/verify/<code>/**.

Recurso	Ruta	Notas
Esquema OpenAPI	**/api/v1/schema/**	drf-spectacular; título “SIGA-INEBI API”, versión 0.1.0.
Documentación interactiva	**/api/v1/docs/**	Swagger UI.
Disponibilidad	**/api/v1/health/**	Público.
Verificación de base	**/api/v1/health/database/**	Ejecuta **SELECT 1**; responde 503 si no hay base.

3.5 Versionado

La API se versiona por prefijo de ruta; la versión actual es **/api/v1/**. Cualquier cambio incompatible en rutas, payloads, respuestas o semántica debe documentarse antes de introducir una nueva versión pública (**AGENTS.md**, regla 3). El esquema publica **VERSION = 0.1.0**, que es la versión del contrato de documentación y no del prefijo.

3.6 Autenticación

La autenticación predeterminada es **SessionAuthentication** con cookies y CSRF; no hay JWT ni tokens de API. La política por defecto es **IsAuthenticated**. La autorización se evalúa en backend por operación con RBAC, permisos atómicos y alcances obligatorios; su detalle está en el capítulo 5.

3.7 Procesos programados y procesamiento diferido

ADR-0009 difiere los trabajos en segundo plano. Lo que existe es un mecanismo deliberadamente simple: comandos de gestión que **cron** invoca y que dejan constancia en **common.TaskRun** y en el logger **siga.tasks**.

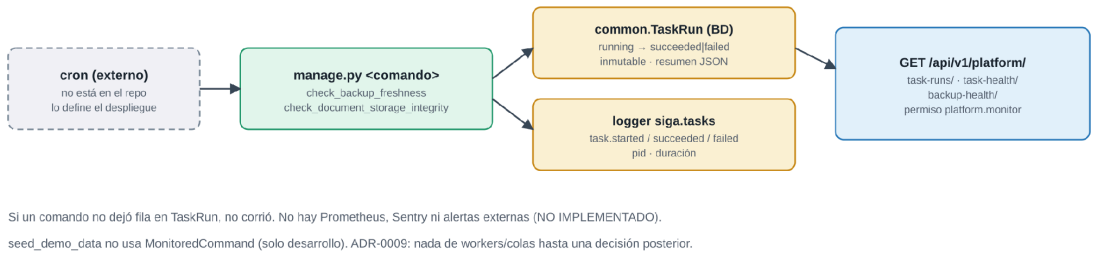


Figura 9. *Tareas programadas, registro de ejecución y endpoints de monitoreo.*

Comando	App	Base	Función
check_document_storage_integrity	**documents**	**MonitoredCommand**	Compara los archivos con su SHA-256 almacenado (**--institution-id**, **--json**).
check_backup_freshness	**common**	**MonitoredCommand**	Verifica que cada pila de respaldo sea más reciente que el RPO (**--json**).

Comando	App	Base	Función
seed_demo_data	**identity**	**BaseCommand**	Crea institución demo, estructura, roles y administrador demo. No se monitorea; solo desarrollo.

3.8 Inconsistencias conocidas de la API

Se documentan para que no se descubran leyendo el código:

Hallazgo	Detalle	Referencia
Token de descarga sin endpoint de emisión	**issue_document_download_token** existe en **documents/services.py** y está probado, pero ninguna vista lo invoca; la API solo expone la validación en **GET records/<id>/download/?token=**.	RF de descarga · sección 5.10
Identificadores mezclados	**students/** direcciona al estudiante por **int pk** y sus subrecursos (notas, contactos, observaciones) por **public_id** UUID.	**students/api/urls.py**
Sin límite de tasa en el login	Solo **document_verification** (20/min por IP) tiene throttle. El login se protege por bloqueo de cuenta (5 fallos, 10 min), no por IP.	RNF-SEG-006
Enums de OpenAPI con nombres por hash	La colisión de enums **status** entre cuatro dominios produce nombres como **Status113Enum**.	**config/settings/base.py**
evaluation anidada en **academics**	Sus rutas cuelgan de **academics/cycles/<id>/evaluation-units/…**.	**academics/api/urls.py**
Lotes síncronos	**grades/bulk/**, **re-enrolments/bulk/** y **official-issuance/batches/** corren dentro de la petición.	ADR-0009 · RNF-REN-003 (Deferred)

4. Frontend

El cliente es React 19 con Vite 7 y React Router 7, con componentes MUI 7 y TanStack Query. **frontend/src/app/navigation.js** es el registro único de los módulos privados: de ahí salen el menú, las rutas y la carga perezosa. Agregar una pantalla es agregar una entrada ahí.

4.1 Estructura, alias y presupuesto de build

Elemento	Detalle
Directorios de dominio	**src/academics**, **app**, **attendance**, **auth**, **cycles**, **dashboard**, **documents**, **enrolments**, **evaluation**, **guardians**, **people**, **reporting**, **shared**, **students**, **teachers**, **test**.
Alias de Vite	Uno por dominio y capa (**@auth**, **@students**, **@academics**, **@layout**, **@ui**, …).
Navegadores objetivo del build	Chrome 107, Edge 107, Firefox 104, Safari 16.
Presupuesto de build	**npm run build** falla si excede: carga inicial 230 000 B gzip, incremento por ruta 20 000 B, página fría 250 000 B, rasterizados 25 000 B.
Servidor de desarrollo	Vite en **0.0.0.0:5173**; proxy de **/api** y **/media** hacia **VITE_BACKEND_PROXY_TARGET** (por defecto **http://localhost:8000**) con **changeOrigin: false**.

4.2 Rutas

routes.jsx define tres rutas públicas y 18 privadas (el panel más 17 módulos en cinco grupos).

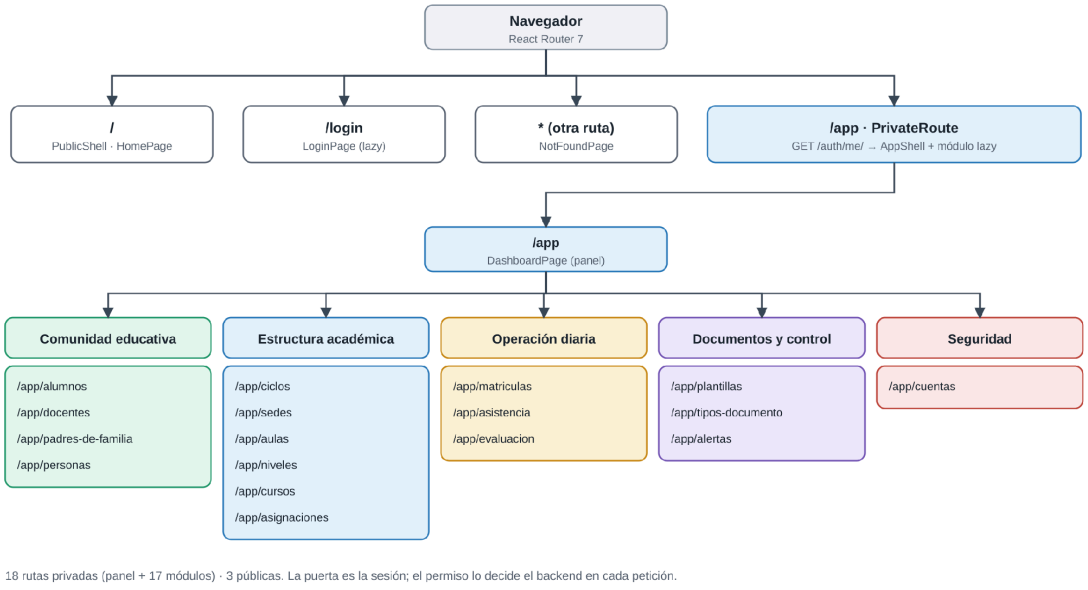


Figura 10. *Mapa de rutas del frontend: públicas, protegidas y módulos por grupo.*

Ruta	Quién entra	Pantalla
/	Cualquiera	**HomePage**, dentro de **PublicShell**.
/login	Cualquiera	**LoginPage** (carga perezosa con **React.lazy**).
/app y el resto de **navigation.js**	Sesión válida	Módulo dentro de **AppShell**.
Cualquier otra	Cualquiera	**NotFoundPage**.

Grupo	Ítem	Ruta	Página
—	Panel	**/app**	**dashboard/DashboardPage**
Comunidad educativa	Estudiantes	**/app/alumnos**	**students/AlumnosPage**
	Docentes	**/app/docentes**	**teachers/DocentesPage**
	Padres de familia	**/app/padres-de-familia**	**guardians/GuardiansPage**
	Personas	**/app/personas**	**people/PersonasPage**
Estructura académica	Ciclo escolar	**/app/ciclos**	**cycles/CyclesPage**
	Sedes	**/app/sedes**	**academics/CampusesPage**
	Aulas	**/app/aulas**	**academics/RoomsPage**
	Niveles	**/app/niveles**	**academics/LevelsPage**
	Cursos	**/app/cursos**	**academics/SubjectsPage**
	Asignaciones docentes	**/app/asignaciones**	**academics/TeachingAssignmentsPage**
Operación diaria	Matrículas	**/app/matriculas**	**enrolments/EnrolmentsPage**
	Asistencia	**/app/asistencia**	**attendance/AttendancePage**
	Evaluación	**/app/evaluacion**	**evaluation/EvaluationPage**
Documentos y control	Plantillas	**/app/plantillas**	**documents/TemplatesPage**
	Tipos de documento	**/app/tipos-documento**	**documents/DocumentTypesPage**
	Alertas	**/app/alertas**	**reporting/AlertsPage**
Seguridad	Cuentas	**/app/cuentas**	**auth/AccountsPage**

Cada módulo privado se carga con **React.lazy**. Pasar el puntero por el ítem del menú llama a **prefetchModule** con el mismo **import**, de modo que el fragmento ya está cargado al hacer clic. Los contadores del menú (**useModuleCounts**) solo se calculan con sesión y aplican a estudiantes, docentes y padres de familia.

4.3 Protección de pantallas

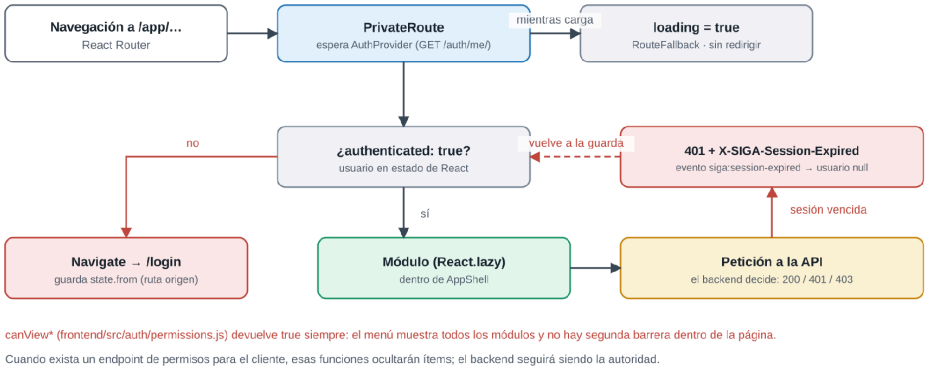


Figura 11. *Guarda de rutas privadas y reacción a una sesión vencida.*

La puerta de la ruta es la sesión, no el permiso atómico. **PrivateRoute** espera a que **AuthProvider** termine de preguntar **GET /auth/me/**; mientras **loading** es verdadero muestra **RouteFallback** y no redirige. Si no hay usuario, manda a **/login** y guarda la ruta de origen en el estado de navegación.

> **Atención. frontend/src/auth/permissions.js** (**canViewAlumnos**, **canViewDocentes**, **canViewPadres**, **canViewSedes**, **canViewNiveles**, **canViewCursos**) devuelve **true** siempre: aún no existe un endpoint que entregue permisos al cliente. Hoy el menú muestra todos los módulos y no hay una segunda barrera dentro de la página. Si alguien abre **/app/alumnos** con sesión, la pantalla se monta y la API responde 401 o 403 si la cuenta no alcanza. La autoridad de acceso es el backend.

4.4 Sesión por cookies

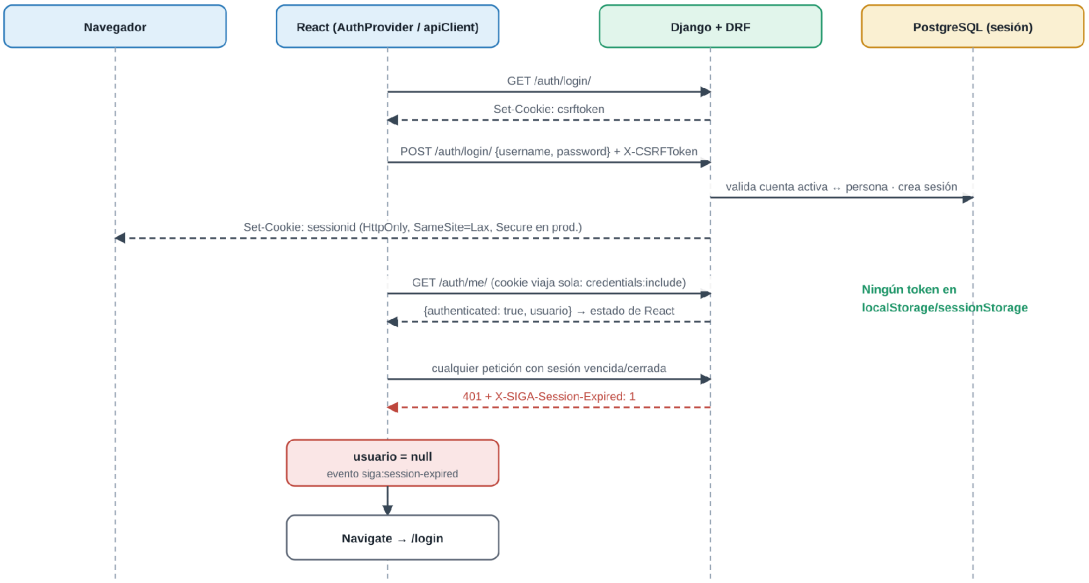


Figura 12. *Secuencia de ingreso, verificación de sesión y vencimiento.*

- Al montar, **AuthProvider** llama **GET /auth/me/**. El usuario vive en el estado de React; al recargar, ese estado se pierde y se vuelve a pedir **me**: la sesión sigue si el navegador envía la cookie.
- El ingreso pide primero **GET /auth/login/** para recibir la cookie CSRF y después **POST /auth/login/** con usuario y contraseña. No hay casilla de “mantener sesión”: la duración la fija el backend (inactividad de 30 minutos por defecto, configurable por rol).
- **logout** y **logoutAll** llaman **/auth/logout/** y **/auth/logout/all/** y borran el usuario del estado.
- Si una respuesta de error trae **X-SIGA-Session-Expired: 1**, **apiClient** dispara el evento **siga:session-expired** y el proveedor deja al usuario en nulo; la siguiente navegación privada cae en **/login**.

4.5 Consumo de API

frontend/src/shared/api/apiClient.js es el único **fetch** de la aplicación. La base es **VITE_API_URL** o **/api/v1**. Toda petición usa **credentials: "include"** y, en métodos que no son **GET**, **HEAD**, **OPTIONS** ni **TRACE**, el encabezado **X-CSRFToken** leído de la cookie **csrftoken**. Los cuerpos **FormData** (fotografías, adjuntos) no fuerzan **Content-Type**.

Los listados paginados aceptan **page**. Un filtro en falso no se envía, salvo **acknowledged=false** en alertas, que es la consulta útil del tablero (**reportingService**, probado en **moduleServices.test.js**). Errores: **apiClient** construye un **Error** con **status** y **payload**, y extrae el texto de las distintas formas de **detail** descritas en la sección 3.3. El Anexo A lista los 142 endpoints que el cliente consume, con sus parámetros.

4.6 Qué no se guarda en el navegador

- No hay token de acceso ni de refresco: la sesión es una cookie **HttpOnly** que JavaScript no puede leer.
- El login no escribe la contraseña ni la sesión en **localStorage** ni en **sessionStorage**. Lo cubre la prueba *does not store password or session token in localStorage on login* en **frontend/src/test/App.test.jsx**.
- Lo único que esa prueba permite en **localStorage** es **mui-mode**, la preferencia de tema de MUI, que no identifica al usuario.
- La única cookie que JavaScript lee es **csrftoken**, con **HttpOnly = False** a propósito, para reenviarla en **X-CSRFToken**. No autentica por sí sola.
- El token de restablecimiento de contraseña que muestra “Cuentas” queda en el estado de React de esa pantalla para copiarlo una vez; no se persiste.

4.7 Componentes y flujos principales por dominio

Dominio	Ruta y pantalla	Flujos y ventanas
Panel	**/ HomePage** · **/app DashboardPage**	Destino tras ingresar; contadores del menú; salud del proceso (**GET /health/**).
Personas	**/app/personas PersonasPage**	Alta, edición y baja lógica de la persona institucional. No abre sesión ni matrícula.
Identidad	**/login LoginPage** · **/app/cuentas AccountsPage**	Desde cuentas se desactiva, se cierran sesiones ajenas y se emite un restablecimiento. **ChangePasswordWindow** cambia la contraseña propia.

Dominio	Ruta y pantalla	Flujos y ventanas
Estudiantes	**/app/alumnos AlumnosPage**	Expediente, incluida la fotografía; notas de salud, observaciones y contactos de emergencia.
Docentes	**/app/docentes DocentesPage**	Los puestos del formulario salen de **POSITION_OPTIONS** en **teachersMock.js**, que copia las opciones del modelo; los registros no son ficticios. La asignación a curso está en **/app/asignaciones**.
Encargados	**/app/padres-de-familia GuardiansPage**	Lista encargados y su vínculo con estudiantes; el expediente sigue en **/app/alumnos**.
Estructura académica	**/app/ciclos**, **sedes**, **aulas**, **niveles**, **cursos**, **asignaciones**	Ciclos: alta, detalle, activación y clonado. Sedes con jornadas. Aulas con estado de servicio. Niveles con grados y plan. Asignación docente, curso y sección.
Matrículas	**/app/matriculas EnrolmentsPage**	Ventanas: lote (**BulkEnrolmentWindow**), historial de movimientos (**MovementHistoryWindow**) y documentos (**EnrolmentDocumentsWindow**).
Asistencia	**/app/asistencia AttendancePage**	Escaneo (**ScanCaptureWindow**; la cámara solo abre en contexto seguro), parámetros de jornada, cierre de sección y presencia. El backend valida credencial y evento.
Evaluación	**/app/evaluacion EvaluationPage**	Captura de notas, excepciones, nota final y promedio en curso. La pantalla no decide la promoción.
Documentos	**/app/plantillas** · **/app/tipos-documento**	Versiones, vista previa y etiquetas de plantilla; catálogo de tipos. La verificación pública del código impreso no tiene pantalla: es **GET /api/v1/documents/verify/<code>/**.
Alertas	**/app/alertas AlertsPage**	**AlertEvaluationWindow** atiende una alerta; **AbsenceThresholdsWindow** edita umbrales. La asistencia que las dispara se registra en **/app/asistencia**.

4.8 Capacidades del backend sin pantalla

Varios endpoints existen y funcionan, pero el frontend no los usa todavía:

- Bitácora de auditoría (**audit/events/**, exportación CSV, declaraciones de retención, trazabilidad de resultados).
- Gestión de roles, asignaciones de rol y catálogo de permisos (**identity/roles/**, **role-assignments/**, **permissions/**).
- Emisión oficial de documentos, entrega, reportes históricos y descarga con token (**documents/official-issuance/…**, **records/…**).
- Credenciales QR (emisión, impresión, revocación y resolución) y justificaciones o permisos de asistencia.
- Monitoreo de plataforma (**platform/task-runs/**, **task-health/**, **backup-health/**).

> **Nota.** Verificar con el equipo si alguna de estas capacidades se ofrece dentro de ventanas modales de las páginas existentes (por ejemplo **CameraPreviewWindow**) antes de darlas por ausentes en la interfaz.

4.9 Pruebas del frontend

Vitest 3.2 con jsdom y Testing Library: 30 archivos en **frontend/src/test/** (por ejemplo **App.test.jsx**, **apiClient.test.js**, **AlumnosPage.test.jsx**, **AttendancePage.test.jsx**, **Sidebar.test.jsx**, **ChangePasswordWindow.test.jsx**, **moduleNavigation.test.jsx**, **useModuleCounts.test.jsx**) y **check-build-budget.test.mjs**. Los umbrales de cobertura son líneas 60, funciones 60, ramas 50 y sentencias 60; el detalle está en la sección 6.6.

5. Seguridad y auditoría

Este capítulo describe los controles que el código implementa hoy, las fuentes que los rigen (**docs/architecture/authorization-model.md**, **data-classification.md**, **audit-strategy.md**, **file-storage-strategy.md**, ADR-0005 y ADR-0006) y, al final, las brechas conocidas. Lo que no está implementado se marca como tal con su requisito o decisión asociada.

5.1 Principios

- **Denegación por defecto:** una vista sin permiso y alcance declarados no da acceso; el permiso se evalúa en cada operación.
- **Múltiples roles por usuario**, con efecto aditivo.
- **Permisos atómicos con alcance obligatorio:** un permiso sin alcance no concede nada.
- **Cuenta siempre vinculada a una persona institucional.**
- **Historia antes que borrado:** desactivación, revocación y estados en lugar de eliminación física (ADR-0006).
- **Lecturas y escrituras sensibles auditables**, con bitácora inmutable a nivel de aplicación.

5.2 Modelo de roles, permisos y alcances

El modelo (ADR-0005) es RBAC con concesiones acotadas por alcance. El diagrama entidad-relación de identidad está en la sección 6.2; aquí se explica cómo funciona en tiempo de ejecución.

Concepto	Implementación real	Detalle
Cuenta	**identity.UserAccount** (**AUTH_USER_MODEL**)	Extiende **AbstractUser**; añade **person** (1:1, **PROTECT**), **status** (**pending**, **active**, **blocked**, **disabled**), **failed_login_attempts** y **locked_until**.
Rol	**identity.Role**	Fila configurable (no un enum): **name**, **slug**, **is_system**, **session_idle_timeout_minutes** (30 por defecto) y **permissions** (M2M).
Permiso atómico	**django.contrib.auth.Permission**	39 códigos definidos en **identity/atomic_permissions.py** y sembrados por migración. El **codename** reemplaza el punto por guion bajo (**student.view_basic** → **student_view_basic**). No existe un modelo **PermissionGrant** propio.
Asignación de rol	**identity.RoleAssignment**	**user**, **role**, **starts_at**, **ends_at**. Vigente cuando **starts_at ≤ ahora** y **ends_at** es nulo o futuro.
Alcance	**identity.ScopeGrant**	Dimensiones nulas: institución, ciclo, grado, sección, materia, asignación docente, estudiante, más **module_key**. Una restricción de base rechaza concesiones activas sin ninguna dimensión.

Cómo se decide un acceso

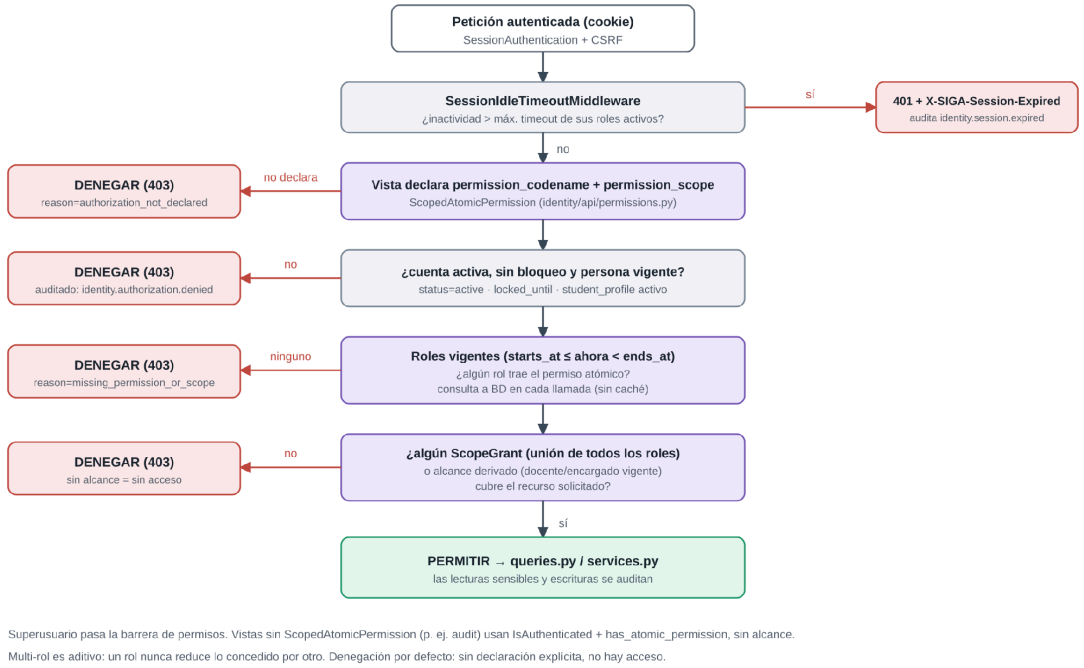


Figura 13. *Decisión de autorización de una petición autenticada.*

- **ScopedAtomicPermission** (**identity/api/permissions.py**) exige que la vista declare **permission_codename** y **permission_scope**. Si falta alguno, deniega y audita **identity.authorization.denied** con **reason=authorization_not_declared**.
- **UserAccount.has_atomic_permission** devuelve falso si la cuenta no está activa, está bloqueada o si la persona tiene un expediente de estudiante no activo. En otro caso consulta la base en cada llamada: no hay caché de permisos en la sesión, de modo que una revocación surte efecto en la siguiente petición.
- **has_scoped_permission** sin alcance devuelve falso. Con alcance delega en **apps.identity.scopes.scope_matches**.
- **Roles múltiples:** los alcances efectivos de todas las asignaciones activas se combinan por **unión**; un rol nunca reduce lo concedido por otro.
- **Alcance derivado:** un docente accede a los estudiantes de las secciones donde tiene una **TeachingAssignment** vigente (ciclo activo, fechas y matrícula activa), y un encargado ve a sus estudiantes por una **StudentGuardianRelation** vigente (solo **student_view_basic**), sin requerir **ScopeGrant** (PD-004 discute su granularidad).
- **Protecciones del último administrador:** no se puede revocar el último rol administrador, retirar **account.create** al último rol que lo concede ni desactivar la última cuenta administradora; están prohibidas la autoasignación y la autorrevocación de roles.

Atención. Algunas vistas (por ejemplo **audit/api/views.py**) usan **IsAuthenticated** más **has_atomic_permission** sin alcance. **docs/architecture/audit-strategy.md** pide consultar la bitácora “con permiso y alcance”; hoy solo se verifica el permiso **audit.read**.

5.3 Cuentas vinculadas a personas

UserAccount.person es un **OneToOneField** a **people.Person** con **on_delete=PROTECT**: no existe cuenta sin persona institucional, una persona tiene como máximo una cuenta y no se puede eliminar una persona que tenga cuenta. **Student**, **Guardian** y **Teacher** también cuelgan 1:1 de **Person**, de modo que una misma persona puede ser docente y encargado sin duplicar sus datos personales.

- **Alta: POST identity/accounts/** crea la cuenta y devuelve un código de activación numérico de 8 dígitos **una sola vez**; solo se guarda su HMAC (**token_digest**).
- **Activación: POST identity/accounts/activate/** (público) con vigencia de 15 minutos y 3 intentos; la reemisión revoca el desafío anterior; la comparación es de tiempo constante.
- **Restablecimiento:** lo emite un administrador (**accounts/<id>/password-resets/**, token **secrets.token_urlsafe(32)**, 30 minutos) y se canjea en **password-resets/consume/**. No hay autoservicio "olvidé mi contraseña" por correo: no existe envío de correo en el código.
- **Desactivación:** cambia **status**; nunca borra la cuenta, y conserva **actor_label** en la bitácora.

5.4 Autenticación, sesión y contraseñas

Control	Valor por defecto	Dónde se configura
Bloqueo por intentos fallidos	5 intentos → bloqueo de 10 minutos	**LOGIN_MAX_FAILED_ATTEMPTS**, **LOGIN_LOCKOUT_MINUTES** (entorno)
Inactividad de sesión	30 min; se aplica el **mayor session_idle_timeout_minutes** entre los roles activos	**DEFAULT_SESSION_IDLE_TIMEOUT_MINUTES** y **Role.session_idle_timeout_minutes**
Vigencia del código de activación	15 min · 3 intentos	**ACCOUNT_ACTIVATION_TTL_MINUTES**, **ACCOUNT_ACTIVATION_MAX_ATTEMPTS**
Vigencia del restablecimiento	30 min	Constante en **identity/services.py**
Longitud mínima de contraseña	10 caracteres	**PASSWORD_MIN_LENGTH**
Validadores de contraseña	Similitud con atributos, longitud mínima, contraseñas comunes	**AUTH_PASSWORD_VALIDATORS**
Hash de contraseña	PBKDF2 (predeterminado de Django)	Solo **settings/test.py** usa MD5, por velocidad de pruebas
Límite de tasa	Solo **document_verification**: 20/min por IP	**DEFAULT_THROTTLE_RATES**

- Al vencer por inactividad, **SessionIdleTimeoutMiddleware** audita **identity.session.expired**, vacía la sesión y responde 401 con **X-SIGA-Session-Expired: 1**; la expiración es deslizante (**set_expiry**).
- Un cambio de contraseña propio (**auth/password/change/**) exige la contraseña vigente y cierra las demás sesiones. Un administrador puede cerrar las sesiones de otra cuenta (**accounts/<id>/sessions/close/**) y cada usuario todas las suyas (**auth/logout/all/**).

- Las credenciales inválidas y la cuenta bloqueada responden 400 con el mismo texto, y el evento **identity.login.denied** no registra la contraseña ni identificadores de cuentas inexistentes.

5.5 Cookies seguras, CSRF, CORS y TLS

Ajuste	development	staging	production
SESSION_COOKIE_HTTPONLY	True	True	True
SESSION_COOKIE_SAMESITE	Lax	Lax	Lax
SESSION_COOKIE_SECURE y **CSRF_COOKIE_SECURE**	False (por entorno)	True	True
CSRF_COOKIE_HTTPONLY	False (el SPA lee el token)	False	False
SECURE_SSL_REDIRECT	No	No	Sí
HSTS	No	No	31 536 000 s + subdominios + preload
DEBUG	**DJANGO_DEBUG** (falso por defecto)	False	False
Validación al arrancar	—	**SECRET_KEY** distinta de la de desarrollo y **ALLOWED_HOSTS** no vacío; PostgreSQL obligatorio	Igual que staging

- **SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")**: la aplicación **no termina TLS**; requiere un proxy inverso que envíe **X-Forwarded-Proto: https**. No hay configuración de proxy en el repositorio.
- CSRF: **CsrfViewMiddleware** activo y **SessionAuthentication** aplica CSRF a métodos inseguros. Orígenes permitidos por **CSRF_TRUSTED_ORIGINS** y **CORS_ALLOWED_ORIGINS** (**CORS_ALLOW_CREDENTIALS = True**).
- La cámara del escaneo QR exige contexto seguro (**isSecureContext**), es decir HTTPS fuera de localhost.
- Orden de middleware: **SecurityMiddleware**, **CorsMiddleware**, sesión, **Locale**, **Common**, **Csrf**, autenticación, **SessionIdleTimeoutMiddleware**, mensajes, **XFrameOptions** y **AuditContextMiddleware**.

Atención. El **.env.example** de backend define **SESSION_COOKIE_SECURE** y **CSRF_COOKIE_SECURE**, pero **base.py** no las lee: ambas dependen de **DJANGO_ENVIRONMENT**. Tampoco hay Content-Security-Policy; solo se aplican las cabeceras predeterminadas de **SecurityMiddleware** y **XFrameOptions**.

5.6 Auditoría

audit.AuditEvent es la única bitácora: reemplaza los tres registros del modelo inicial (**AuditLog**, **SensitiveReadLog**, **DeniedAttemptLog**) con un solo modelo cuya columna **action** distingue el tipo de evento.

Aspecto	Implementación
Contenido del evento	**actor** (FK a **UserAccount**, **SET_NULL**), **actor_label** (usuario persistente), **action**, **resource**, **resource_identifier**, **ip_address** y **context** JSON con **path**, **user_agent**, **reason** y **changes** (antes/después).
Servicio	**apps.audit.services.record_event**; el **AuditContextMiddleware** aporta IP (**REMOTE_ADDR**), agente y ruta.
Minimización	**sanitize_context** elimina claves **password**, **token**, **access_token**, **refresh_token**, **secret**, **private_key**, **credential(s)** y **cookie(s)**.
Inmutabilidad	**save()** sobre un evento existente y **delete()** lanzan **RuntimeError**; **QuerySet.update()/delete()** también; el admin de Django no permite borrar.
Lecturas sensibles	**record_sensitive_read** en estudiantes y asistencia. Se audita solo cuando la petición nombra a un único estudiante (no en agregados). Ejemplos: **students.emergency_contact.detail_read**, **documents.document.read**, **attendance.day_status.read**, **audit.result_trace.viewed**.
Denegaciones	Ejemplos: **identity.authorization.denied**, **identity.login.denied**, **documents.document.read_denied**, **attendance.event.capture_denied**, **evaluation.grade_write_denied**.
Escrituras	Unas 120 acciones distintas: cuentas, roles, matrículas, notas, credenciales QR, emisión de documentos, alertas, exportaciones y declaraciones de retención.
Consulta	**GET audit/events/** (paginado, filtros por usuario, capacidad, acción y fechas; permiso **audit.read**) y **GET audit/events/export/** (CSV, que a su vez se audita como **audit.export.created**).

5.7 Minimización y clasificación de datos

docs/architecture/data-classification.md define cuatro niveles: público, interno, restringido y sensible especial.

Dato	Clasificación	Tratamiento real
Identificador del QR	Restringido	32 bytes aleatorios (**secrets**), sin datos personales; una credencial activa por estudiante (índice único parcial); revocable con historial.
Fotografía	Restringido	**Student.photo** y **Teacher.photo**; fuera de la base, en **MEDIA_ROOT**.
Contactos de emergencia	Restringido	Lectura individual auditada.
Notas de salud	Sensible especial	Lectura individual auditada; acceso restringido por permiso y alcance (PD-005 fija la política de observaciones disciplinarias).
Documentos de expediente	Sensible especial o restringido según tipo	Metadatos inmutables, SHA-256 y descarga con token breve.
Justificaciones	Sensible especial	Adjunto con checksum y almacenamiento fuera de la base.
Respaldos	Sensible especial	Sin cifrado implementado (sección 5.11).

Dato	Clasificación	Tratamiento real
Bitácora	Restringido / sensible	Contexto saneado; sin contraseñas ni tokens.
Parámetros institucionales	Interno	—

- No se usan datos reales en desarrollo ni pruebas (**AGENTS.md**, regla 6); **seed_demo_data** produce datos ficticios.
- La pantalla de escaneo solo confirma lo mínimo necesario; PD-006 deja abierto qué datos son visibles en la credencial impresa.

5.8 Protección de secretos

- **.env** no se versiona; **.env.example**, **backend/.env.example** y **frontend/.env.example** solo contienen valores de desarrollo (**DJANGO_SECRET_KEY=change-me-in-local-env**, contraseña de base de desarrollo, **DEMO_ADMIN_PASSWORD=admin**). No sirven para producción.
- **base.py** usa una clave insecura por defecto solo para desarrollo; staging y producción lanzan **ImproperlyConfigured** si sigue siendo esa o si **ALLOWED_HOSTS** está vacío.
- Pre-commit: **detect-private-key**, **check-added-large-files** y **detect-secrets** con **.secrets.baseline**, además de **ruff** y **prettier**.
- CI: **bandit -q -r apps config** y **pip-audit** (con una lista temporal de 7 vulnerabilidades ignoradas, documentada en **docs/development/security-tooling.md**); **pr-validation** rechaza **.env*** (salvo **.env.example**), llaves y certificados, bases SQLite, **.sql**, **media/uploads/** y rutas que contengan **secret**, **credential**, **backup** o **dump**.
- En producción los secretos viven solo en el proveedor de despliegue y no se registran cookies, enlaces de restablecimiento ni variables (**docs/development/secure-deployment.md**).
- **SECURITY.md** recoge los principios y los pendientes de endurecimiento: gestión de secretos por ambiente, política formal de retención, rotación de credenciales y respuesta a incidentes.

5.9 Archivos y datos sensibles

- **Binarios fuera de la base:** PostgreSQL guarda metadatos y una **storage_key**; el archivo va a **default_storage** (**FileSystemStorage**) bajo **MEDIA_ROOT** (**backend/media**, no versionado).
- **Nombre no derivable: storage_key = documents/<student.public_id>/<token aleatorio><extensión>**; el nombre original no forma parte de la ruta.
- **Validación de subida (`validate_document_upload`):** archivo no vacío; extensión **.pdf**, **.jpg**, **.jpeg** o **.png**; MIME declarado **application/pdf**, **image/jpeg** o **image/png**; máximo 10 MiB (**DOCUMENT_MAX_UPLOAD_SIZE_BYTES**); nombre normalizado.
- **Integridad:** SHA-256 al guardar; **check_document_storage_integrity** recalcula y compara (verificación periódica programable con **cron**); si falla la persistencia se elimina el archivo ya escrito.
- **Inmutabilidad:** los metadatos de **DocumentRecord** no se modifican tras crearse; una nueva versión crea otro registro (**version_group_id**, **version_number**, **supersedes**).
- **Capacidad:** crecimiento de referencia de 2 GiB por ciclo (**DOCUMENT_STORAGE_GROWTH_PER_CYCLE_BYTES**), consultable en **documents/storage-consumption/**.

- **Verificación pública: GET documents/verify/<code>/** usa un código de **secrets.token_urlsafe(24)** con límite de 20 solicitudes por minuto por IP.

Descarga con token de vigencia breve

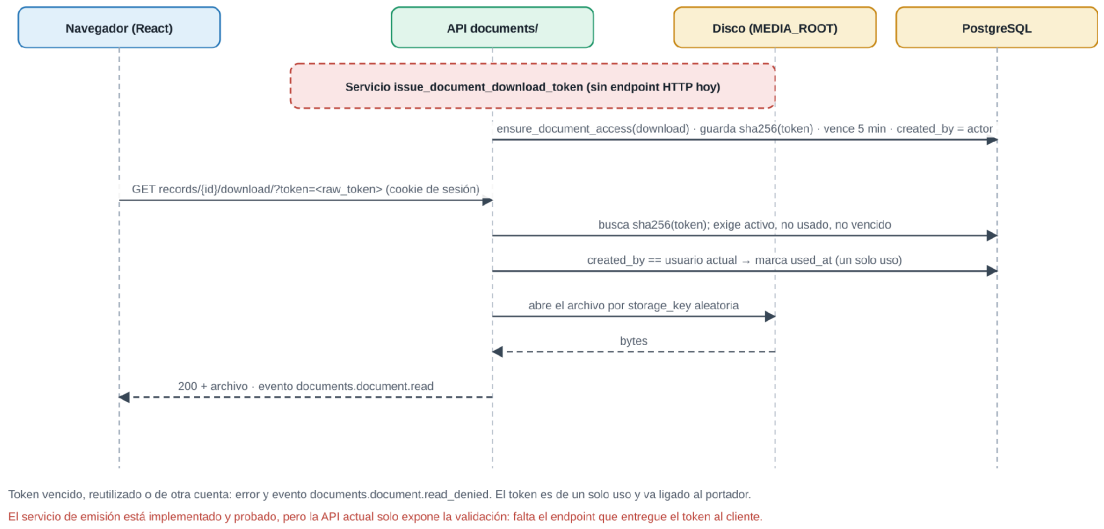


Figura 14. *Descarga de un documento con token de un solo uso ligado al portador.*

El token dura 5 minutos, es de un solo uso, se guarda como SHA-256 y solo lo puede consumir la cuenta que lo pidió. Por eso el enlace no es una URL estática del archivo ni sirve compartido. La emisión requiere pasar **ensure_document_access(access_type="download")**, que comprueba permiso y alcance.

5.10 Flujo de captura y datos de asistencia

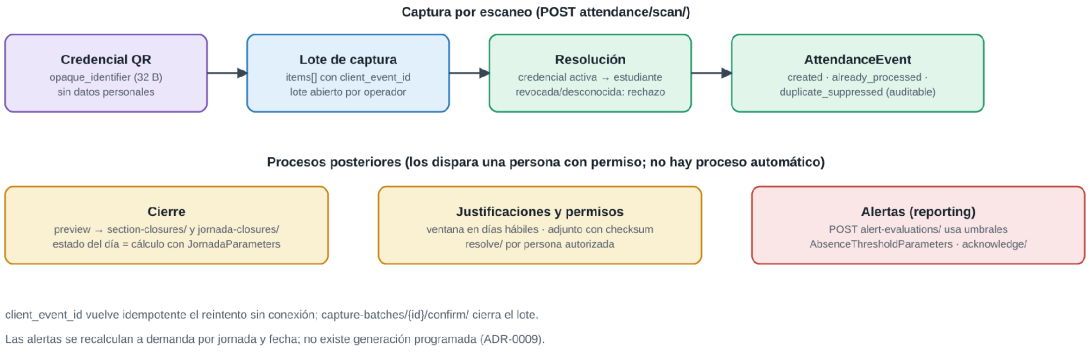


Figura 15. *Captura por escaneo QR y procesos posteriores.*

El QR no lleva datos personales: la app resuelve el identificador a un estudiante solo si la credencial está activa. **client_event_id** (índice único parcial) vuelve idempotente el reintento sin conexión; un escaneo repetido dentro de **duplicate_suppression_minutes** se registra como rechazo auditable, no como movimiento. Las credenciales se revocan por cambio de permanencia y se restauran al reabrirla.

5.11 Brechas de seguridad conocidas

Brecha	Estado actual	Referencia
Autenticación multifactor	No implementada	Sin requisito verificado
Límite de tasa en el login por IP	No implementado; solo bloqueo por cuenta	RNF-SEG-006 cubre solo la verificación pública
Restablecimiento de contraseña por correo	No implementado (sin envío de correo)	—
Retención y purga de bitácora y documentos	No implementada; solo declaración de retención	PD-003 · RNF-LEG-001
Cifrado de respaldos y del disco de archivos	No implementado en scripts ni en compose	**data-classification.md** clasifica los respaldos como sensible especial
Inmutabilidad de la bitácora en PostgreSQL	Solo capa de aplicación	RF-BIT-005
Validación del contenido real de archivos	Solo extensión, MIME declarado y tamaño; sin *magic bytes* ni antivirus	**file-storage-strategy.md** (normalización de imágenes)
Content-Security-Policy	No implementada	—
Alcance en la consulta de bitácora	Solo permiso, sin alcance	**audit-strategy.md**
Bloqueo de ramas y revisiones obligatorias	No aplicadas por limitación del plan de GitHub	Sección 6.8
Endpoint que entrega el token de descarga	El servicio existe; la API no lo expone	Sección 3.8
IP real tras el proxy	Se registra **REMOTE_ADDR**, no **X-Forwarded-For**	**AuditContextMiddleware**

6. Base de datos y calidad

PostgreSQL 16 es la base de referencia (ADR-0003). El ORM de Django es la vía principal de acceso a datos y las migraciones son la única fuente de esquema. Este capítulo presenta los modelos conceptual, lógico y físico, el estado real del diccionario de datos y de los scripts finales, y luego la estrategia de pruebas, la integración continua y el flujo de trabajo del equipo.

6.1 Modelo conceptual

docs/architecture/initial-data-model.md define el modelo inicial de la fase fundacional en seis agrupaciones. El código real conserva esas agrupaciones, con 69 modelos propios repartidos en 12 apps.

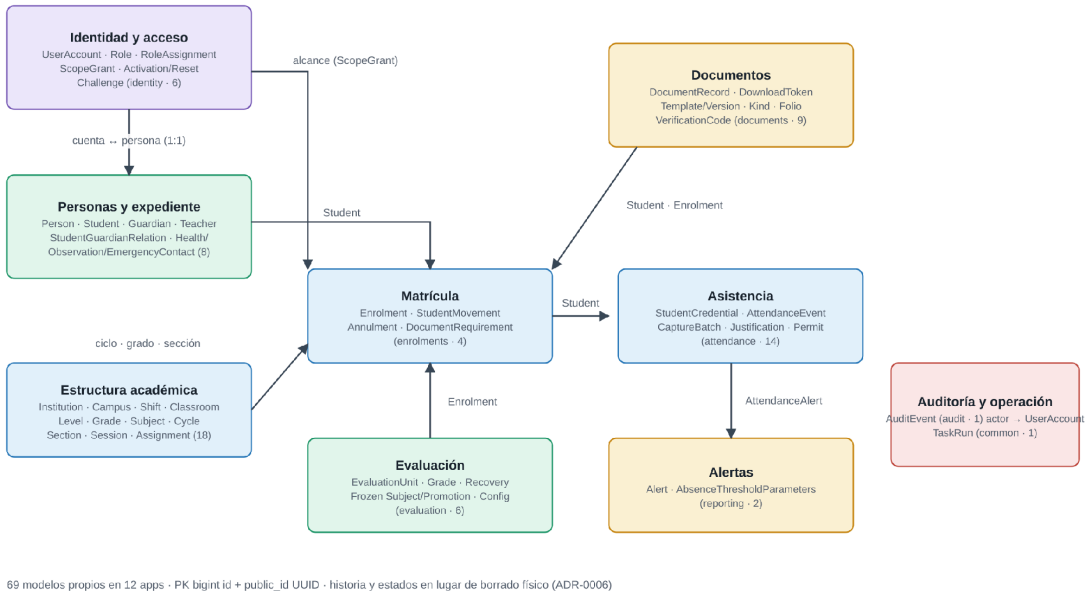


Figura 16. *Modelo conceptual final: agrupaciones de entidades y relaciones principales.*

Agrupación	Entidades (nombres reales del código)	Idea central
Identidad y acceso	**UserAccount**, **Role**, **RoleAssignment**, **ScopeGrant**, **ActivationChallenge**, **PasswordResetChallenge**	Cuenta ligada a persona; permisos con alcance obligatorio.
Personas y expediente	**Person**, **Student**, **Guardian**, **Teacher**, **StudentGuardianRelation**, **EmergencyContact**, **StudentHealthNote**, **StudentObservation**	**Person** es el ancla 1:1 de cuenta, estudiante, encargado y docente.
Estructura y ciclo	**Institution**, **Campus**, **Shift**, **Classroom**, **Level**, **Grade**, **Subject**, **AcademicCycle**, **GradeOffering**, **Section**, **CurriculumPlan**, **ClassSession**, **TeachingAssignment**, …	Un solo ciclo activo por institución; oferta por ciclo, jornada y grado.
Matrícula	**Enrolment**, **StudentMovement**, **StudentMovementAnnulment**, **EnrolmentDocumentRequirement**	Una matrícula activa por estudiante; los movimientos se anulan, no se borran.

Agrupación	Entidades (nombres reales del código)	Idea central
Evaluación	**EvaluationUnit**, **Grade** (calificación), **RecoveryGrade**, **CaptureExceptionGrant**, **Cycle/GlobalEvaluationConfig**, **FrozenSubjectResult**, **FrozenPromotionResult**	Los resultados congelados se corrigen con un nuevo registro y motivo.
Asistencia y alertas	**StudentCredential**, **AttendanceEvent**, **CaptureBatch**, **ControlPoint**, **JornadaParameters**, **Justification**, **AttendancePermit**, **AttendanceAlert**, **Alert**, …	El QR es un identificador opaco; el estado del día se calcula.
Documentos	**DocumentRecord**, **DocumentDownloadToken**, **DocumentTemplate(Version)**, **DocumentKind**, **OfficialFolio**, **DocumentVerificationCode**, …	Binarios fuera de la base; versiones inmutables.
Auditoría y operación	**AuditEvent**, **TaskRun**	Registros de solo agregar.

Invariantes que el modelo conceptual declara y el código hace cumplir: historia primero (estados y vigencias en lugar de borrado físico), correcciones sensibles que agregan evidencia, y binarios fuera de la base (la base guarda metadatos y una referencia).

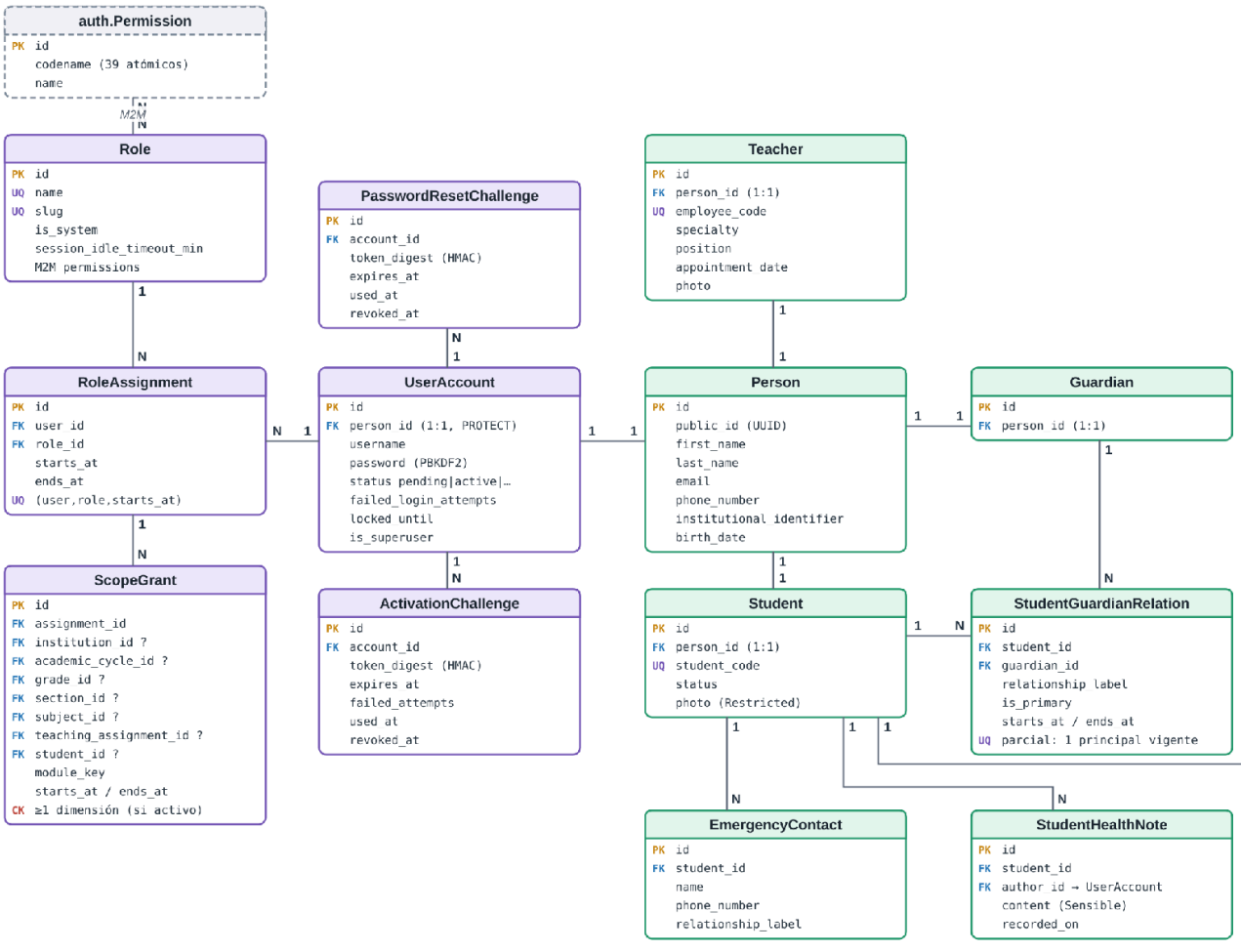
Diferencias entre el modelo inicial y el código

El modelo inicial es un documento de diseño; la fuente de verdad son los modelos Django. Quien compare ambos encontrará estas equivalencias:

Modelo inicial (documento)	Modelo real (código)
SchoolCycle, **CycleState**	**AcademicCycle** con el campo **status**
AcademicSubarea, **StudyPlan**	**Subject**, **CurriculumPlan**, **LevelSubject**
GuardianLink	**Guardian** + **StudentGuardianRelation**
Enrollment	**Enrolment** (ortografía británica)
Credential	**StudentCredential**
CaptureShift	**Shift**
AttendanceDayStatus	No existe como tabla: el estado del día se calcula con **JornadaParameters**
DocumentType, **FileObject**, **DocumentVersion**, **GeneratedDocument**, **Template**	**DocumentKind**, **DocumentRecord** (con **version_group_id**, **version_number**, **supersedes**), **DocumentTemplate**, **DocumentTemplateVersion**
AuditLog, **SensitiveReadLog**, **DeniedAttemptLog**	Un único **AuditEvent** con **action** y **resource**
Session	No es un modelo propio: **django_session** de Django
Permission, **PermissionGrant**	**auth.Permission** + M2M **Role.permissions**

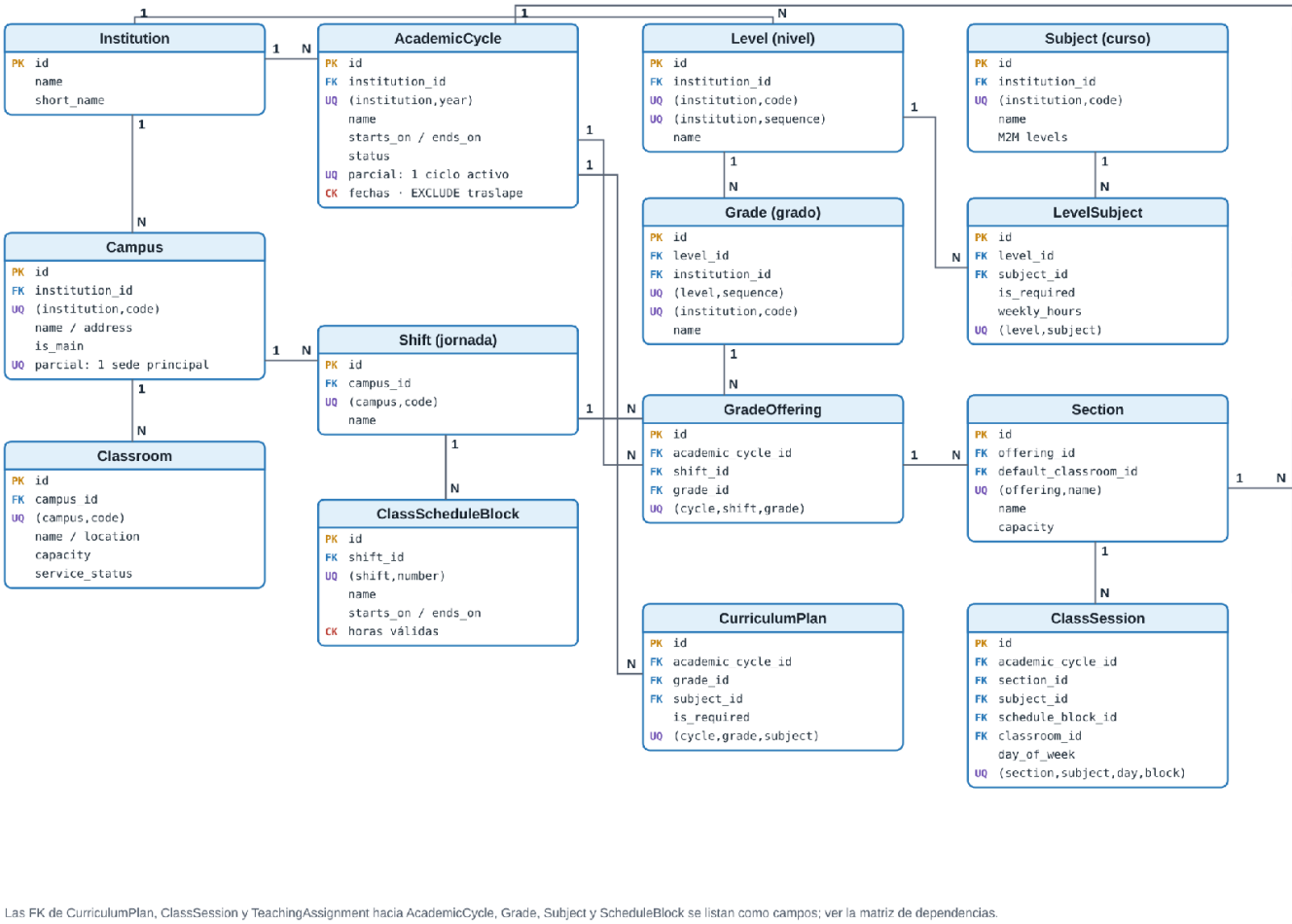
6.2 Modelo lógico y diagrama entidad-relación final

Las páginas siguientes (en horizontal) contienen el diagrama entidad-relación final del sistema en siete vistas por dominio. Cada entidad muestra sus claves primarias (PK), foráneas (FK), unicidades (UQ) y restricciones de verificación (CK); los recuadros punteados son entidades de otro dominio incluidas solo como referencia. Las relaciones se rotulan con su cardinalidad. El Anexo B es el inventario completo de las 69 tablas.



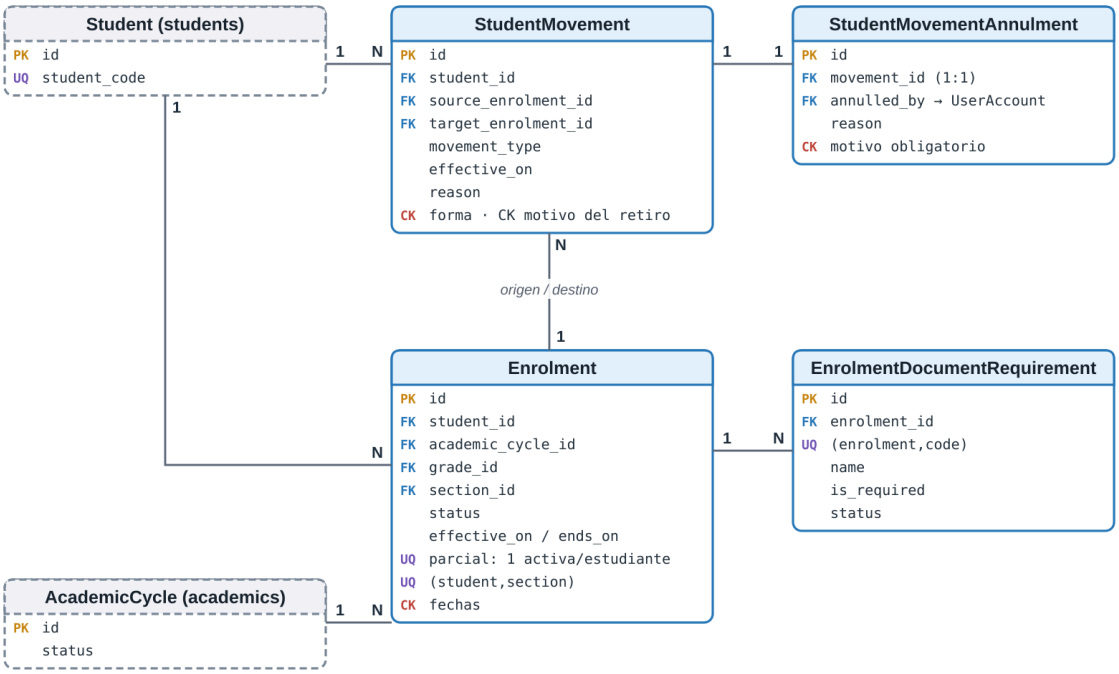
? = FK nula. Cada ScopeGrant exige al menos una dimensión (CK identity_scope_grant_has_dimension).

Figura 17. *ER · Identidad, personas, expediente y auditoría*



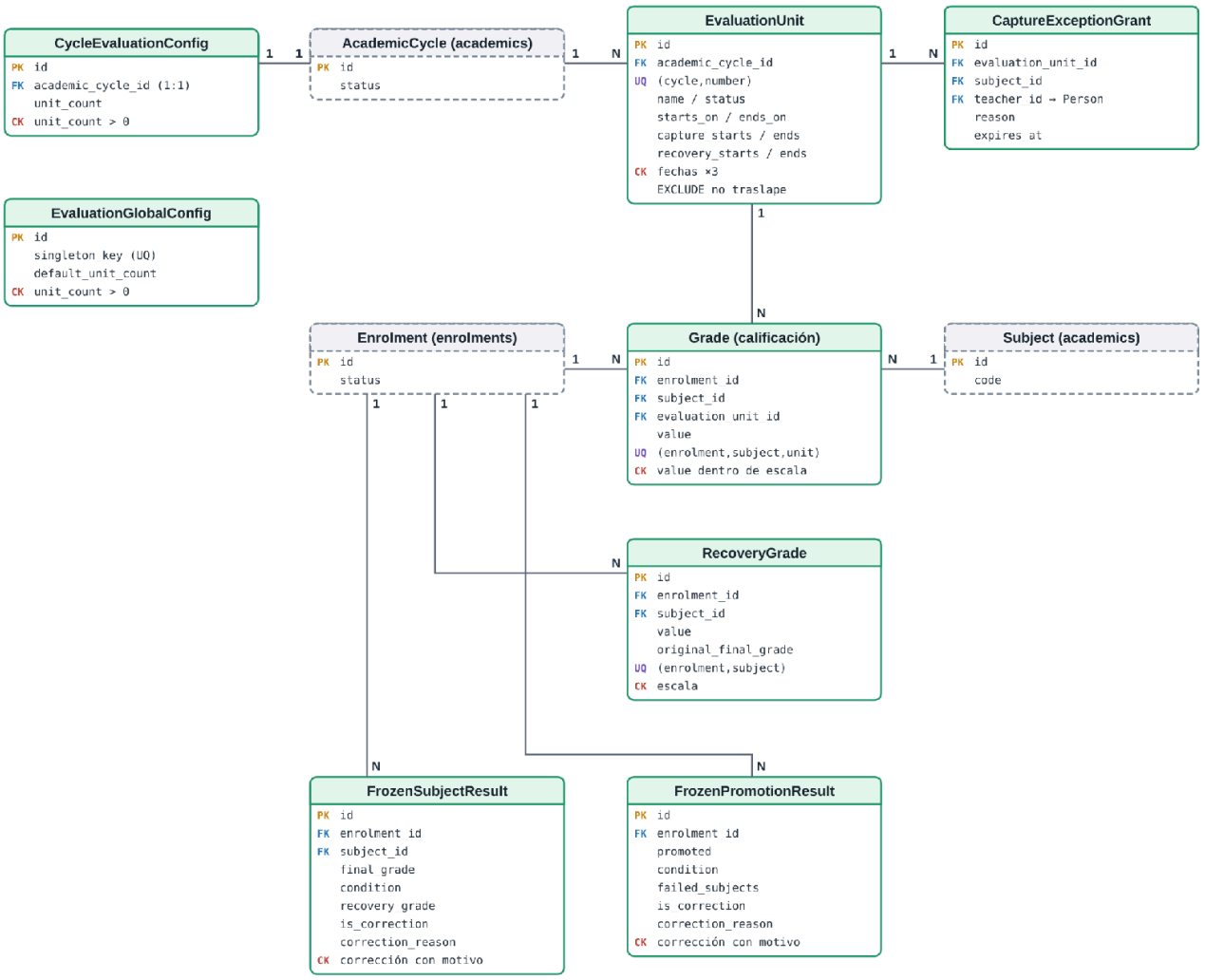
Las FK de CurriculumPlan, ClassSession y TeachingAssignment hacia AcademicCycle, Grade, Subject y ScheduleBlock se listan como campos; ver la matriz de dependencias.

Figura 18. *ER · Estructura académica.*



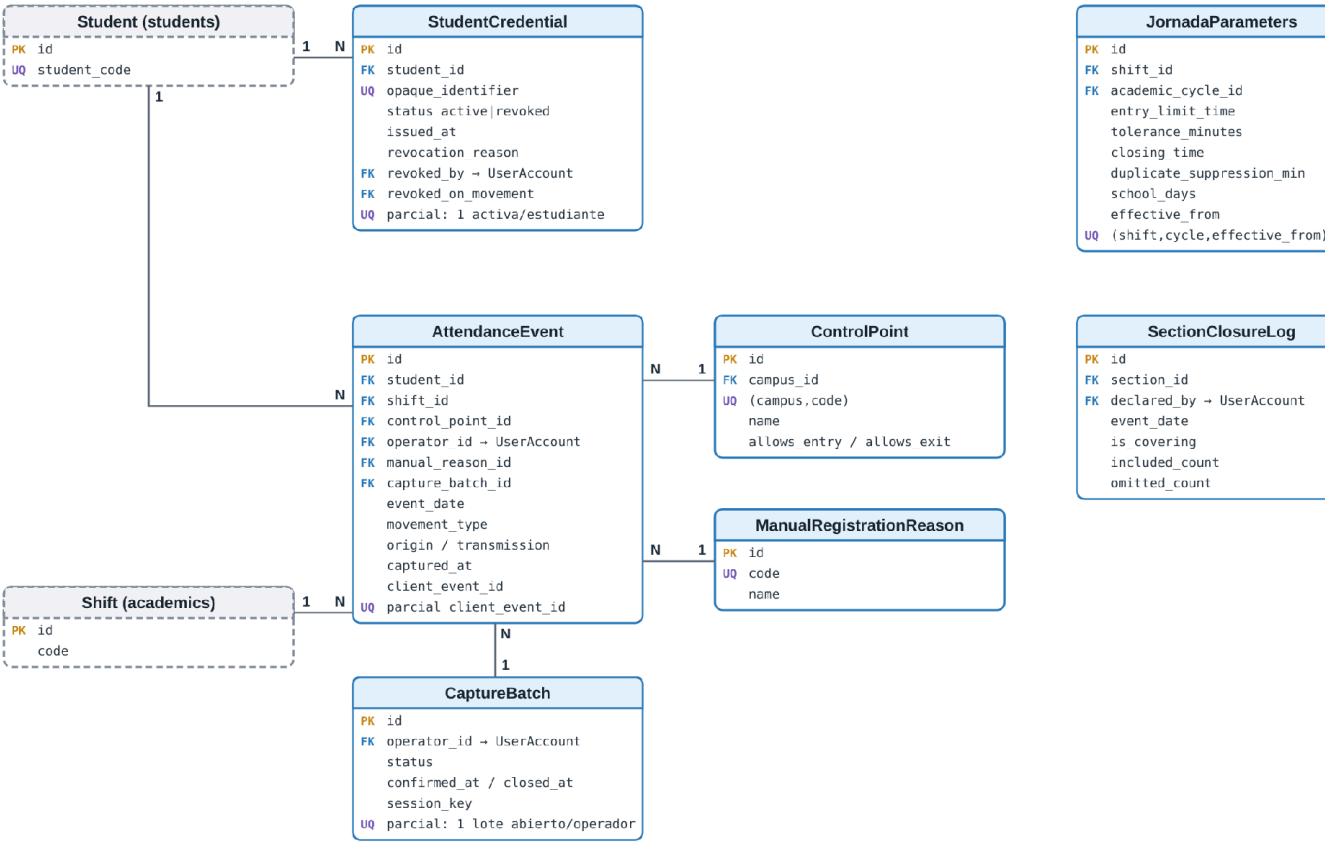
Grade y Section provienen de academics; una matrícula activa por estudiante se garantiza con un índice único parcial.

Figura 19. *ER · Matrícula y movimientos.*



Los resultados congelados no se editan: una corrección crea un nuevo registro con is_correction=true y motivo obligatorio. EvaluationGlobalConfig es un singleton.

Figura 20. *ER · Evaluación y resultados congelados.*



Un QR contiene únicamente opaque_identifier (32 bytes aleatorios). AttendanceDayStatus del modelo inicial NO existe como tabla: el estado del día se calcula.

Figura 21. *ER · Credencial y captura de asistencia.*

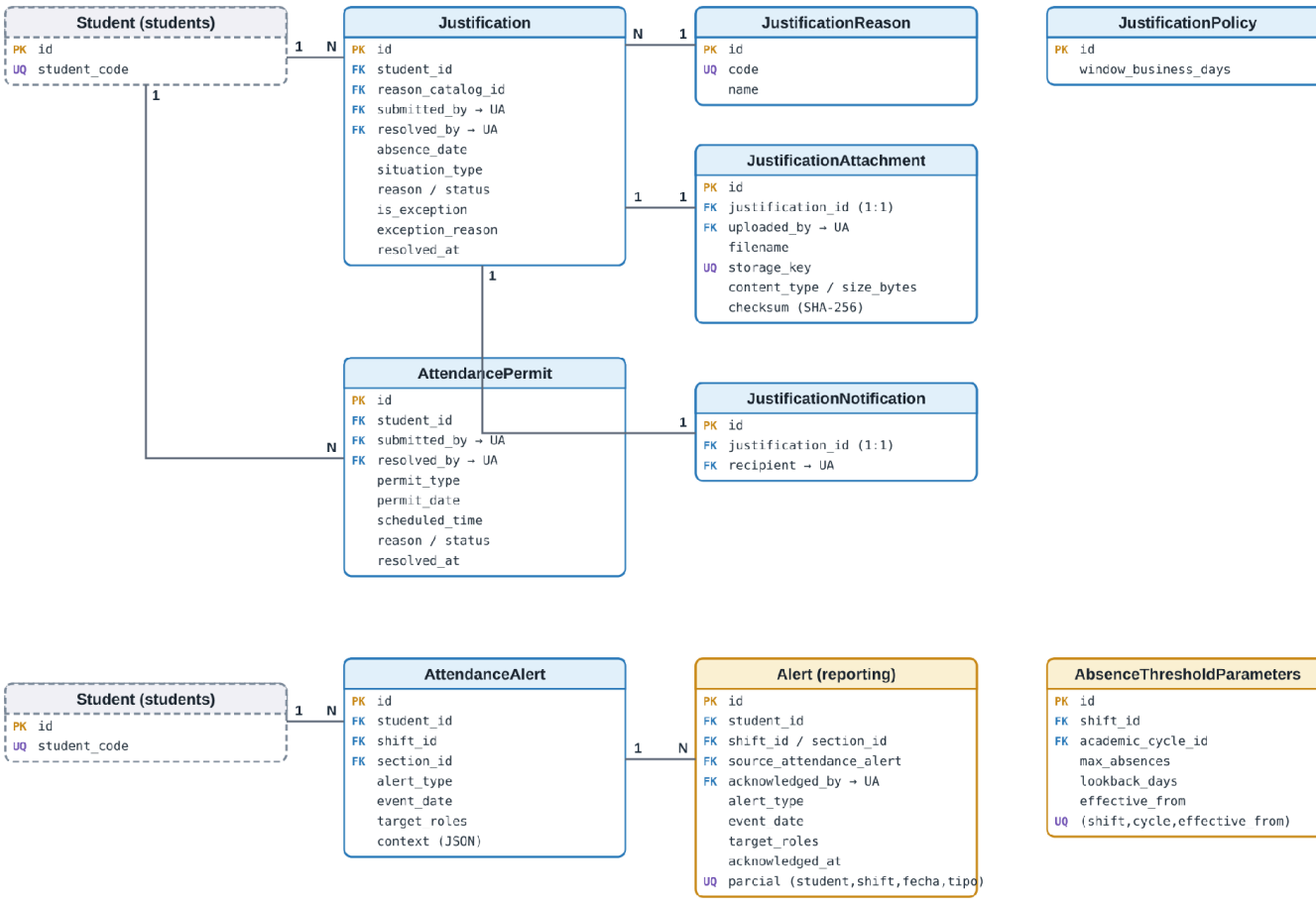


Figura 22. *ER · Justificaciones, permisos y alertas.*

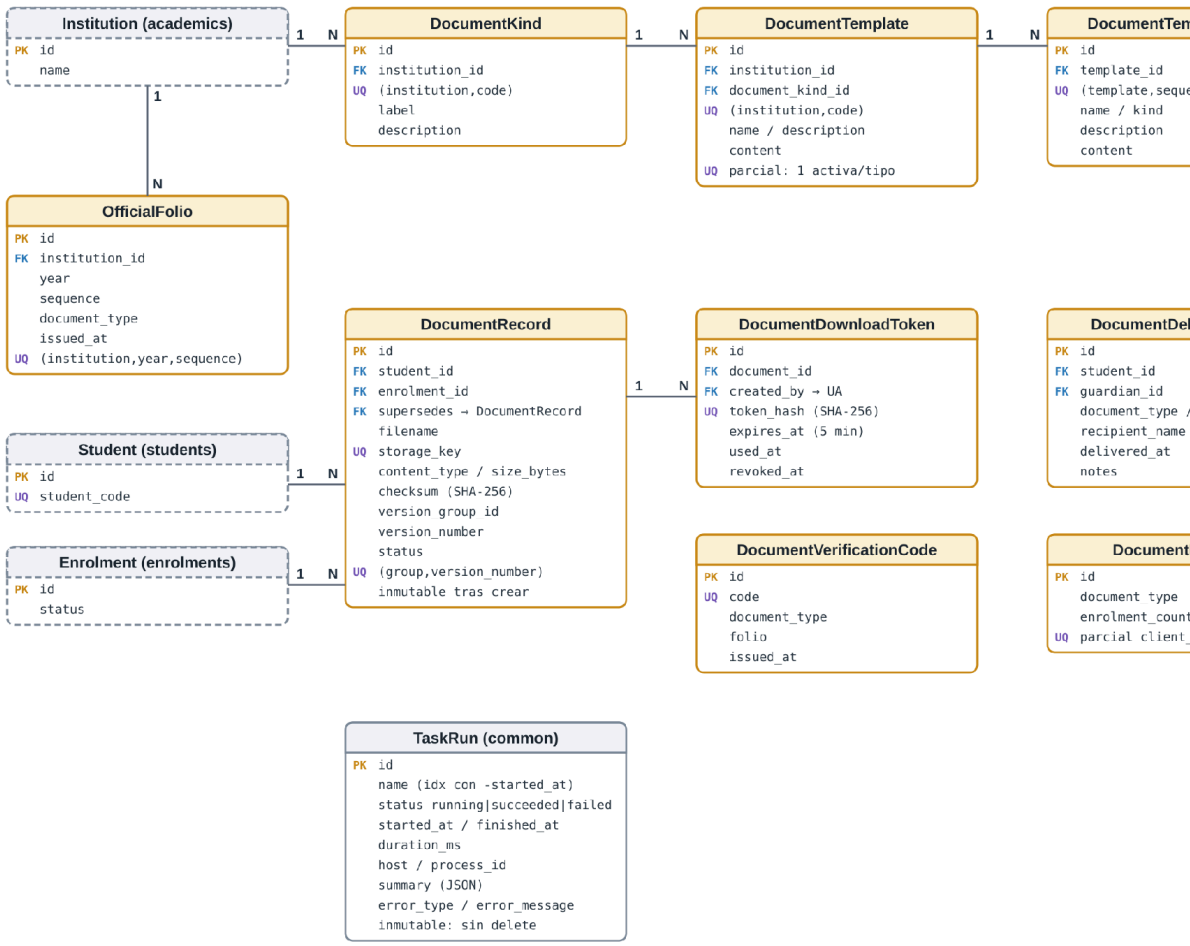


Figura 23. *ER · Documentos, verificación y monitoreo de tareas.*

6.3 Modelo físico

El modelo físico es el que generan las migraciones de Django sobre PostgreSQL 16. Rasgos relevantes:

- **Nombres:** el predeterminado de Django, **<app>_<modelo en minúsculas>** (por ejemplo **academics_academiccycle**); ninguna tabla define **db_table**. Hay dos modelos **Grade** con tablas distintas: **academics_grade** (grado escolar) y **evaluation_grade** (calificación).
- **Claves: id** bigint (**DEFAULT_AUTO_FIELD = BigAutoField**) más **public_id** UUID único y no editable en casi todos los modelos, que es lo que la API expone.
- **Columnas comunes (`TimeStampedModel`): public_id**, **created_at**, **updated_at** e **is_active** (base del borrado lógico, ADR-0006). **UserAccount** hereda de **AbstractUser** y no de esa base.
- **Tiempo:** instantes como **timestamptz**; la sesión de Django trabaja en UTC (**USE_TZ**) y el clúster PostgreSQL se fija a la zona del establecimiento (**timezone**, **log_timezone**, **TZ**, **PGTZ**). Fechas de efecto y días escolares se derivan del reloj local.
- **Conexiones: CONN_MAX_AGE = 60** s (**DATABASE_CONN_MAX_AGE**).
- **Restricciones específicas de PostgreSQL: ExclusionConstraint** (**django.contrib.postgres**) para impedir el traslape de fechas en **AcademicCycle**, **EvaluationUnit** y **TeachingAssignment**. El traslape de horas de **ClassScheduleBlock** no se resuelve con exclusión (no hay rango de tiempo) sino en **academics/services.py** bajo bloqueo de fila.
- **Índices:** los de claves foráneas y unicidad, más **task_run_name_recent_idx** (**name**, **-started_at**) en **common_taskrun**.

Invariantes garantizados por la base

Los índices únicos parciales trasladan reglas de negocio a la base, de modo que ni un error de aplicación puede violarlas:

Restricción	Regla
unique_active_cycle_per_institution	Un solo ciclo con **status = 'active'** por institución.
unique_main_campus_per_institution	Una sola sede principal por institución.
unique_active_enrolment_per_student	Una matrícula activa por estudiante.
unique_active_student_credential	Una credencial QR activa por estudiante; **opaque_identifier** único global.
unique_open_capture_batch_per_operator	Un lote de captura abierto por operador.
unique_attendance_event_client_event_id	Idempotencia de eventos enviados sin conexión (**client_event_id** no vacío).
students_one_current_primary_guardian	Un encargado principal vigente por estudiante.
unique_active_alert_per_student_day_type	Una alerta activa por estudiante, jornada, día y tipo.
unique_active_document_template_per_kind_per_institution	Una plantilla activa por tipo de documento e institución.
unique_document_batch_run_client_batch_id	Idempotencia de lotes de emisión.

Restricción	Regla
identity_scope_grant_has_dimension	Todo alcance activo tiene al menos una dimensión.
student_withdrawal_requires_reason, ***_correction_requires_reason**	Retiros y correcciones exigen motivo.

6.4 Diccionario de datos y scripts finales de base de datos

6.5 Migraciones

App	Migraciones	Contenido notable
academics	17	Niveles MINEDUC precreados (**0006**), catálogo académico (**0002**), fundación del ciclo (**0004**).
attendance	23	La app con más evolución.
audit	1	
common	1	**TaskRun**.
documents	15	Versiones inmutables (**0009**), **DocumentKind** (**0013**).
enrolments	7	Reglas de duplicados de matrícula (**0004**).
evaluation	4	
identity	13	Siembra del catálogo de permisos atómicos (**0004**, **0005**, **0007**, **0008**, **0010**, **0012**, **0013**), restricción de dimensión (**0006**), tiempo de sesión por rol (**0009**).
people	2	
reporting	1	
students	7	
teachers	2	
Total	**93**	Varias migraciones usan **RunPython** para sembrar catálogos.

Reglas de **docs/development/database-migrations.md**: migraciones pequeñas y revisables, compatibles con PostgreSQL, sin mezclar refactor grande de datos sin plan de reversión, con preservación de historia y vigencias, una migración por cambio coherente y documentación de efectos sobre índices, bloqueos y tiempo. CI verifica que no falten migraciones (**makemigrations --check --dry-run**) y **backend/tests/migrations/** prueba datos y reglas de esquema.

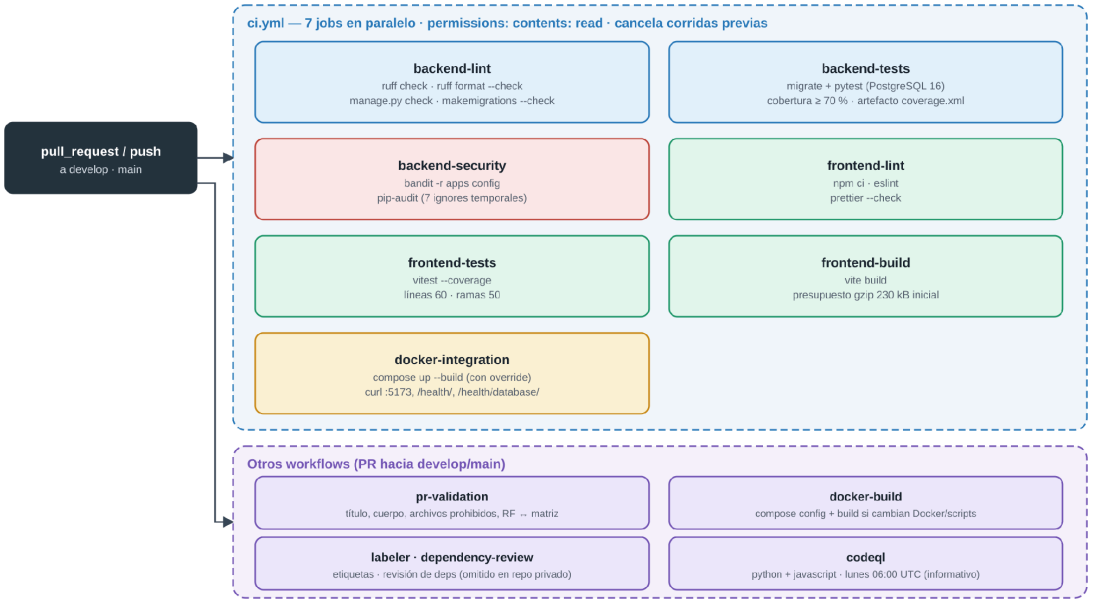
6.6 Estrategia de pruebas



Figura 24. *Mapa de pruebas: dónde se valida cada capa y con qué comando.*

Aspecto	Detalle
Backend	pytest 8.4 + pytest-django 4.11 + pytest-cov 6.2; factory_boy y Faker. Configuración en **backend/pyproject.toml**: **--strict-markers**, cobertura sobre **apps** y **config/api** con **umbral de 70 %** (la corrida falla si baja). Marcadores: **unit**, **integration**, **api**, **permissions**, **slow**, **postgres**, **migration**, **security**.
Ubicación	Todas las pruebas están en **backend/tests/** (81 archivos: unit 27, api 28, integration 18, migrations 4, permissions 4).
PostgreSQL obligatorio	**config.settings.test** aborta con “Test settings require DATABASE_ENGINE=postgresql.” Las dos pruebas del clúster de respaldo se omiten si el cliente **pg_dump** del host no coincide en versión mayor.
Guardas de arquitectura	Pruebas unitarias que verifican los límites de ADR-0008 (sección 3.2).
Frontend	Vitest 3.2 + jsdom + Testing Library: 30 archivos. Umbrales: líneas 60, funciones 60, ramas 50, sentencias 60. **npm run build** añade el presupuesto de tamaño.
Docker	**compose.test.yml** levanta **db-test** (PostgreSQL 16 en **tmpfs**, efímero) y **backend-test**, que ejecuta **migrate --noinput && pytest**. **make ci-local** reproduce el CI: construye imágenes, corre pruebas, lint, cobertura y build del frontend, **ruff**, **makemigrations --check**, **bandit** y **pip-audit**, y destruye el stack de prueba.
TDD	Por defecto en reglas de negocio, permisos, alcances, cálculos, servicios y APIs; cada bug deja una prueba de regresión (**docs/development/testing-strategy.md**, **tdd-guide.md**).
No existe	Pruebas de extremo a extremo en navegador (no hay Playwright ni Cypress en **package.json** ni en CI) ni pruebas de carga; PD-010 y PD-001 dependen de medir esas cifras.

6.7 Integración continua y checks



Checks obligatorios previstos: backend-lint, backend-tests, backend-security, frontend-lint, frontend-tests, frontend-build, docker-integration, pr-validation.

Hoy NO se exigen: rulesets y protección de ramas devuelven 403 en el plan del repositorio privado (configure-rulesets.sh solo imprime el plan).

Python 3.11 · Node 22 · postgres:16-alpine · actions/checkout@v4, setup-python@v5, setup-node@v4.

Figura 25. *Workflows de GitHub Actions y checks previstos.*

Workflow	Cuándo corre	Qué valida
ci.yml	PR y push a **develop**/**main**; manual	Siete jobs: **backend-lint** (ruff, **manage.py check**, **makemigrations --check**), **backend-tests** (migrate + pytest, cobertura ≥ 70 %, artefacto **backend-coverage**), **backend-security** (bandit + pip-audit), **frontend-lint** (eslint + prettier), **frontend-tests** (Vitest con cobertura), **frontend-build** (Vite + presupuesto) y **docker-integration** (levanta el stack y prueba **:5173**, **/api/v1/health/** y **/api/v1/health/database/**).
pr-validation.yml	PR a **develop**/**main**	Título **tipo(scope): descripción**, cuerpo con Issue/Relacionado/Justificación, archivos prohibidos, aviso si se cambian más de 1 500 líneas y exigencia de tocar **traceability-matrix.md** si el cuerpo cita un RF o RNF.
docker-build.yml	PR con cambios en Compose, Dockerfiles, requirements, **package*.json** o **scripts/**	**docker compose config** y **docker compose build**.
codeql.yml	PR, push y lunes 06:00 UTC	Análisis de Python y JavaScript. En repositorio privado solo imprime un mensaje informativo.
dependency-review.yml	PR	Revisión de dependencias (falla en severidad alta). Omitido en repositorio privado.
labeler.yml	PR	Etiquetas según **.github/labeler.yml**.

Workflow	Cuándo corre	Qué valida
gga-review.yml	Solo manual	Marcador experimental: termina sin acción si no hay **GGA_API_KEY**.

.github/dependabot.yml propone actualizaciones semanales de npm (**/frontend**), pip (**/backend**) y GitHub Actions, con un máximo de 5 PR abiertos.

> **Atención.** El job **docker-integration** levanta el stack con **compose.override.yml** (depuración y datos demo). No hay ninguna prueba sobre una imagen de producción, porque no existe. CI tampoco corre en pull requests encadenados hacia una rama distinta de **develop** o **main**.

6.8 Estrategia de ramas y pull requests

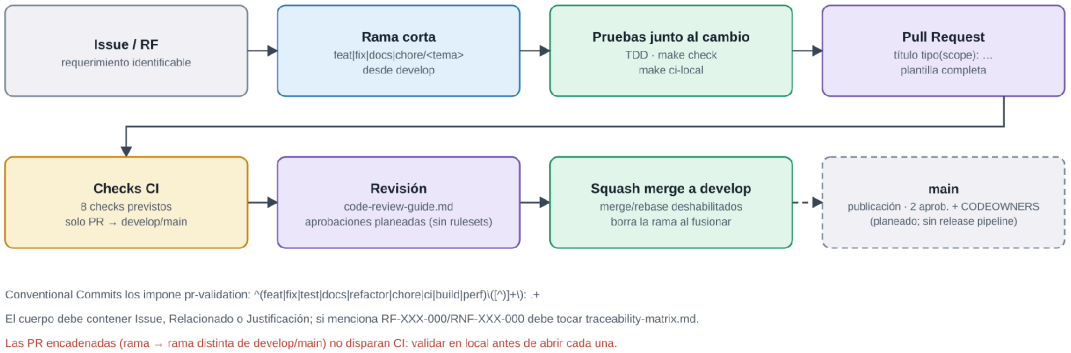


Figura 26. *Flujo de trabajo desde el requisito hasta la fusión.*

- **Ramas: develop** (rama por defecto) y **main**. Ramas de trabajo cortas **feat/<tema>**, **fix/<tema>**, **docs/<tema>**, **chore/<tema>**, con un cambio pequeño y trazable (**docs/development/git-workflow.md**).
- **Commits y títulos de PR:** Conventional Commits **tipo(scope): descripción** con **feat**, **fix**, **test**, **docs**, **refactor**, **chore**, **ci**, **build** o **perf**. Solo lo impone **pr-validation**; **git-workflow.md** no lo menciona. Ejemplo real: **feat(academics): RF-AUL-004 advertencia de capacidad del aula (#527)**.
- **Plantilla de PR:** resumen, problema, solución, issue, RF relacionados, tipo de cambio, evidencia, pruebas locales (**make check**, **make ci-local**), pruebas Docker, migraciones, variables de entorno, seguridad, datos sensibles, documentación, compatibilidad (PostgreSQL, Docker, local), riesgos, plan de reversión y lista de verificación.
- **Fusión:** squash merge; merge commit y rebase deshabilitados; se borra la rama al fusionar; auto-merge deshabilitado.
- **Revisión: docs/development/code-review-guide.md** ordena revisar primero riesgos de negocio, autorización y privacidad, ausencia y calidad de pruebas, violaciones de ADR y trazabilidad.

Control	Plan
Rulesets de **develop**	PR obligatorio, 1 aprobación, descartar aprobaciones obsoletas, conversaciones resueltas, historial lineal, sin force-push ni borrado
Rulesets de **main**	Igual, con 2 aprobaciones y **CODEOWNERS**
Checks obligatorios	**backend-lint**, **backend-tests**, **backend-security**, **frontend-lint**, **frontend-tests**, **frontend-build**, **docker-integration**, **pr-validation**
CODEOWNERS	Revisores por área

La inspección del 2026-07-29 (**docs/development/github-configuration-report.md**) muestra que GitHub devuelve **403 Upgrade to GitHub Pro or make this repository public** para rulesets y protección de ramas en el repositorio privado, y **403 Advanced Security must be enabled** para *code scanning*. Es una limitación del plan, no una decisión.

6.9 Definition of Done

docs/development/definition-of-done.md define seis condiciones para dar un cambio por terminado. Existen además **definition-of-ready.md**, **tdd-guide.md** y **code-review-guide.md**.

#	Condición	Cómo se comprueba
1	Requerimiento referenciado.	**pr-validation** exige Issue/Relacionado/Justificación; la plantilla pide los RF.
2	Pruebas verificables agregadas o actualizadas.	CI: pytest y Vitest con umbrales de cobertura.
3	Trazabilidad actualizada.	**pr-validation** exige tocar **traceability-matrix.md** si se cita un RF/RNF.
4	Documentación actualizada si cambia un contrato o una regla.	Revisión; lista de verificación de la plantilla.
5	Sin secretos ni datos reales.	**detect-secrets**, **detect-private-key**, guardia de archivos de **pr-validation**.
6	No romper ADR vigentes (o actualizar el ADR).	Revisión humana y guardas de arquitectura en pruebas.

6.10 Trazabilidad de requisitos

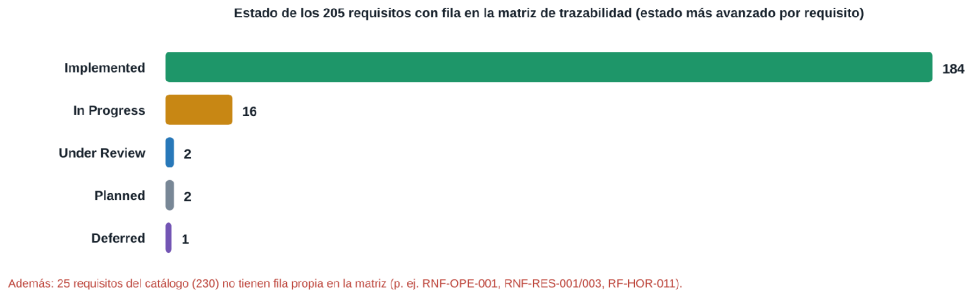


Figura 27. *Estado de los requisitos según la matriz de trazabilidad.*

El catálogo (**docs/requirements/requirements.json**) contiene 230 requisitos: 195 funcionales y 35 no funcionales (176 *Debe*, 44 *Debería*, 10 *Podría*). La matriz (**traceability-matrix.md**) tiene 219 filas que cubren 205 requisitos distintos. Tomando el estado más avanzado de cada requisito: 184 implementados (163 RF y 21 RNF), 16 en progreso, 2 en revisión (RF-AUT-001, RF-CTA-003), 2 planificados (RNF-PRI-003, RNF-RES-002) y 1 diferido (RNF-REN-003, ADR-0009).

7. Operación técnica

Este capítulo reúne los comandos útiles y seguros. Donde existe una guía canónica no se copia su contenido: se enlaza (sección 7.11). Cada comando se marca por su efecto: **[lee]** no modifica nada, **[escribe]** crea o modifica datos o archivos, **[destruye]** puede perder datos.

7.1 Requisitos, versiones y puertos

Elemento	Valor
Python / Django / DRF	3.11 (**python:3.11-slim**) · Django 5.2.17 · DRF 3.17.2 · drf-spectacular 0.28.0
Node / Vite / React	22 (**node:22-alpine**) · Vite 7 · React 19
PostgreSQL	16 (**postgres:16-alpine**); el cliente de la imagen backend es **postgresql-client-16**
Puertos por defecto	frontend **5173** (**FRONTEND_PORT**), backend **8000** (**BACKEND_PORT**), PostgreSQL **5432** (**POSTGRES_PORT**); **vite preview** usa 4173
Zona horaria	**America/Guatemala** (**TIME_ZONE**, RNF-LOC-001)
Variables de Compose (**.env**)	**COMPOSE_PROJECT_NAME**, **POSTGRES_DB**, **POSTGRES_USER**, **POSTGRES_PASSWORD**, **POSTGRES_PORT**, **TIME_ZONE**, **BACKEND_PORT**, **FRONTEND_PORT**, **DEMO_ADMIN_***, **DJANGO_LOG_LEVEL**, **TASK_LOG_LEVEL**

> **Atención.** El **Makefile** fija **DATABASE_PORT=5432** en los objetivos locales. Si su PostgreSQL local no escucha en 5432 (por ejemplo, un **.env** con **POSTGRES_PORT=5433**), sobrescriba **DATABASE_PORT** al invocar **make** o use los comandos con Docker.

7.2 Instalación

Con Docker (recomendado)

```
cp .env.example .env           # [escribe] solo valores de desarrollo
docker compose config          # [lee]     valida la composición
docker compose up --build      # [escribe] construye e inicia db, backend y frontend
# frontend  http://localhost:5173   backend  http://localhost:8000/api/v1/docs/
```

El arranque aplica migraciones y, con **compose.override.yml**, siembra datos demo y un administrador (**DEMO_ADMIN_USERNAME**/**DEMO_ADMIN_PASSWORD** de **.env.example**, ambos **admin**). Solo para desarrollo.

Local sin contenedores para el código

```
sh scripts/setup-local.sh                 # [escribe] .env, backend/.venv y npm install
docker compose up -d db                   # [escribe] PostgreSQL de Compose
```

```
cd backend
export DATABASE_ENGINE=postgresql DATABASE_NAME=siga_inebi DATABASE_USER=siga_inebi \
       DATABASE_PASSWORD=siga_inebi_dev_password DATABASE_HOST=127.0.0.1 DATABASE_PORT=5432
./.venv/bin/python manage.py migrate        # [escribe]
./.venv/bin/python manage.py runserver 127.0.0.1:8000
cd ../frontend && npm ci && VITE_API_URL=http://127.0.0.1:8000/api/v1 npm run dev
```

DATABASE_ENGINE=sqlite permite un arranque rápido, pero las pruebas, el staging y la producción exigen PostgreSQL. Si el intérprete carece de **_ssl**, cree el entorno con **/usr/bin/python3** (**docs/development/troubleshooting.md**).

7.3 Comandos por tarea

Tarea	Comando	Efecto
Ver objetivos disponibles	`make help`	[lee]
Lint (backend y frontend)	`make lint` · `make format-check`	[lee]
Formatear	`make format`	[escribe] reformatea archivos
Pruebas del backend	`make test-backend` · `test-backend-unit` · `test-backend-integration`	[lee] usa la base configurada; exige PostgreSQL
Pruebas del frontend	`make test-frontend` · `cd frontend && npm run test:coverage`	[lee]
Cobertura	`make coverage`	[escribe] `coverage.xml`, `frontend/coverage`
Compilar el frontend	`cd frontend && npm run build`	[escribe] `frontend/dist`; falla si excede el presupuesto
Seguridad	`make security`	[lee] bandit + pip-audit
Verificar migraciones	`make migrations-check`	[lee] `makemigrations --check --dry-run`
Crear migraciones	`make migrations`	[escribe] crea archivos de migración
Aplicar migraciones	`make migrate`	[escribe] modifica el esquema
Validación completa local	`make check`	[escribe] lint, formato, pruebas, cobertura, migraciones y `manage.py check`
Reproducir CI	`make ci-local`	[escribe] levanta y destruye el proyecto `siga_inebi_test` (`down -v` solo de ese proyecto)
Ver estado del stack	`docker compose ps` · `docker compose logs -f`	[lee]
Apagar el stack	`docker compose down` (`make down`)	[escribe] conserva los volúmenes

Tarea	Comando	Efecto
Borrar volúmenes	`docker compose down -v`	**[destruye]** elimina `postgres_data`
Reiniciar la base de desarrollo	`scripts/reset-development-db.sh`	**[destruye]** pide confirmación y hace `down -v`
Limpiar cachés	`make clean`	[escribe] pide confirmación; borra cachés, `frontend/dist` y cobertura

Atención. make check invoca el objetivo **build**, que es **docker compose build**, no **npm run build**: la compilación del frontend se valida en el job **frontend-build** del CI y en **make ci-local**.

7.4 Migraciones

- Verificar sin escribir: **make migrations-check** o **docker compose exec backend python manage.py makemigrations --check --dry-run**.
- Aplicar: **make migrate** o **docker compose exec backend python manage.py migrate**. El contenedor backend ya ejecuta **migrate --noinput** al arrancar.
- Crear: **make migrations**; revíselas antes de confirmarlas y respete las reglas de la sección 6.5.
- Toda migración que altere datos históricos debe tener plan de reversión y validarse en PostgreSQL.

7.5 Docker

- El contenedor backend corre como usuario no root **app**. El backend espera a la base con **scripts/wait-for-postgres.sh** (**pg_isready**).
- **docker compose config** y **docker compose build** son seguros y son lo que ejecuta **docker-build.yml**.
- Pruebas aisladas: **docker compose -f compose.yml -f compose.test.yml run --rm backend-test** (base efímera en **tmpfs**).
- Los puertos y la contraseña de base salen de **.env**; no versione ese archivo.

7.6 Lint, formato y build

- Backend: **ruff check** y **ruff format --check** (línea de 100, Python 3.11, reglas **E,F,I,B,C4,DJ,SIM,UP**, sin **migrations**), **manage.py check**.
- Frontend: **npm run lint** (ESLint 9 con **import-x**, **jsx-a11y**, **react-hooks**), **npm run format:check** (Prettier).
- Pre-commit (**.pre-commit-config.yaml**): higiene de archivos, **detect-private-key**, **check-added-large-files**, **ruff**, **prettier** y **detect-secrets**.
- Build del frontend: **npm run build** ejecuta **vite build** y luego **scripts/check-build-budget.mjs**.

7.7 Respaldo y recuperación

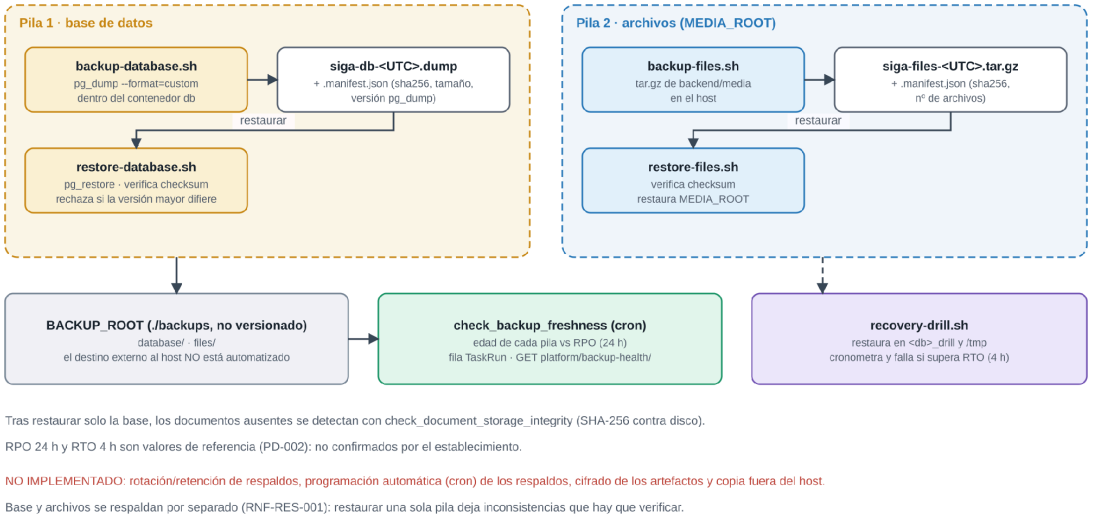


Figura 28. *Pilas de respaldo, verificación de frescura y simulacro de recuperación.*

Base y archivos son pilas independientes (RNF-RES-001). Cada artefacto lleva un **.manifest.json** con SHA-256, tamaño y marca de tiempo UTC; los scripts de restauración verifican el checksum y se niegan a continuar si no coincide, y detectan una diferencia de versión mayor de PostgreSQL antes de escribir.

Acción	Comando	Efecto
Respaldar ambas pilas	`make backup`	[escribe] `BACKUP_ROOT/database/siga-db-<UTC>.dump` y `files/siga-files-<UTC>.tar.gz`
Solo base o solo archivos	`make backup-database` · `make backup-files`	[escribe] la base se respalda dentro del contenedor db
Comprobar RPO	`make backup-check`	[lee] `check_backup_freshness`; deja una fila en `TaskRun`
Restaurar la base	`make restore-database`	**[destruye]** sobrescribe la base destino con el respaldo más reciente
Restaurar archivos	`make restore-files`	**[destruye]** sobrescribe `MEDIA_ROOT`
Simulacro	`make recovery-drill`	[escribe] restaura en `<db>_drill` y en `/tmp`; cronometra y falla si excede el RTO
Integridad de documentos	`python manage.py check_document_storage_integrity`	[lee] compara archivos contra SHA-256

Procedimiento de recuperación

1. Confirmar el alcance: solo base, solo archivos o ambos. Restaurar una pila deja inconsistencias con la otra.

2. Detener el tráfico de escritura (apagar **backend** y **frontend**).
3. Ubicar el respaldo en **BACKUP_ROOT** y comprobar que existe su **.manifest.json**.
4. Restaurar la base (**make restore-database** o **scripts/backup/restore-database.sh <ruta.dump>**); el script valida checksum y versión de **pg_dump**.
5. Restaurar los archivos (**make restore-files** o **restore-files.sh <ruta.tar.gz>**).
6. Aplicar migraciones pendientes y arrancar el backend.
7. Ejecutar **check_document_storage_integrity**: los documentos con estado *missing* revelan desajustes entre base y disco.
8. Verificar **GET /api/v1/health/database/**, entrar con una cuenta de prueba y revisar **platform/backup-health/**.

Variable	Valor por defecto	Uso
`BACKUP_ROOT`	`./backups`	Raíz de los artefactos; no versionada.
`DATABASE_BACKUP_DIR`, `FILES_BACKUP_DIR`	`$BACKUP_ROOT/database` y `/files`	Subcarpetas por pila.
`DATABASE_NAME`, `DATABASE_USER`, `DATABASE_HOST`, `DATABASE_PORT`	`siga_inebi`, `siga_inebi`, `localhost`, `5432`	Conexión de los scripts.
`PGPASSWORD` o `DATABASE_PASSWORD`	—	Nunca se imprime.
`MEDIA_ROOT`	`backend/media`	Origen y destino de la pila de archivos.
`DRILL_DATABASE_NAME`	obligatoria	El simulacro se niega a correr sin ella.
`RECOVERY_POINT_OBJECTIVE_HOURS`, `RECOVERY_TIME_OBJECTIVE_HOURS`	24 y 4	RPO y RTO de referencia (PD-002).

No implementado. No están implementados: rotación o retención de respaldos, programación automática (**cron**) de los respaldos, cifrado de los artefactos y copia fuera del host. La cadencia sugerida es respaldo diario fuera de la jornada, **backup-check** después y un simulacro antes de cada entrega y al menos una vez por ciclo. En el entorno de desarrollo el RTO medido fue de 1 segundo, con una base sin volumen real: no es una cifra representativa.

7.8 Monitoreo y salud

- Salud pública: **GET /api/v1/health/** y **GET /api/v1/health/database/**.
- Con permiso **platform.monitor**: **GET /api/v1/platform/task-runs/** (filtros **name**, **status**), **task-health/** y **backup-health/**.
- Registro de tareas: fila de **common.TaskRun** y logger **siga.tasks** (**TASK_LOG_LEVEL**, hereda **DJANGO_LOG_LEVEL**).
- No hay Prometheus, Sentry ni alertas externas; la observabilidad es **TaskRun**, logs de consola y el admin de Django (solo lectura para **TaskRun**).

7.9 Qué falta para un despliegue expuesto

El repositorio documenta cómo debe configurarse producción pero no lo automatiza. Antes de exponer el sistema faltan, como mínimo:

Pendiente	Detalle
Servidor de aplicación	Las imágenes ejecutan **runserver** y el servidor de desarrollo de Vite; falta un servidor WSGI/ASGI y una imagen estática del frontend.
Proxy TLS	Debe terminar HTTPS y enviar **X-Forwarded-Proto: https**; no hay configuración de proxy versionada.
Secretos	**DJANGO_SECRET_KEY**, contraseña de base, **ALLOWED_HOSTS**, orígenes CORS/CSRF: se inyectan en el proveedor de despliegue.
Programación de tareas	**cron** para **check_backup_freshness**, **check_document_storage_integrity** y los respaldos.
Entrega continua	**docs/development/release-process.md** declara el proceso formal como pendiente; no hay pipeline de despliegue.
Retención y cifrado	PD-003 y la política de cifrado de respaldos siguen abiertos.

7.10 Problemas frecuentes

Síntoma	Causa y acción
ModuleNotFoundError: No module named '_ssl'	El Python del entorno carece de SSL; recree el venv con **/usr/bin/python3**.
connection to server at "127.0.0.1", port 5432 failed	La base no está arriba o el puerto difiere: **docker compose ps**, verifique que **db** esté *healthy* y revise **DATABASE_*** y **POSTGRES_PORT**.
El frontend no llama a la API correcta	Revise **VITE_API_URL** (Docker: **/api/v1** con proxy de Vite; local: **http://127.0.0.1:8000/api/v1**).
Test settings require DATABASE_ENGINE=postgresql.	Es esperado: las pruebas exigen PostgreSQL.
Las pruebas de respaldo se omiten	El **pg_dump** del host no coincide en versión mayor con el servidor (16); use el contenedor.
npm install reporta vulnerabilidades	Ejecute **npm audit** y resuélvalo en una iteración controlada (**troubleshooting.md**).
El PR es rechazado por **pr-validation** por una ruta	La guardia rechaza rutas con **secret**, **credential**, **backup** o **dump** (PD-011): renombre el archivo; no se modifica el workflow.

7.11 Guías canónicas

Estas guías existen y son la fuente de verdad de cada tema; este manual no las duplica.

Tema	Documento
Instalación local	**docs/development/local-setup.md** · **docs/development/onboarding.md**

Tema	Documento
Docker	**docs/development/docker-setup.md** · **docs/development/docker-testing.md**
Pruebas y TDD	**docs/development/testing-strategy.md** · **docs/development/tdd-guide.md**
Migraciones	**docs/development/database-migrations.md**
Flujo de trabajo y revisión	**docs/development/git-workflow.md** · **code-review-guide.md** · **definition-of-ready.md** · **definition-of-done.md**
Seguridad	**docs/development/security-tooling.md** · **docs/development/secure-deployment.md** · **SECURITY.md**
Respaldo y recuperación	**docs/architecture/backup-and-recovery.md**
Monitoreo	**docs/architecture/operations-monitoring.md**
Autorización, auditoría y datos	**docs/architecture/authorization-model.md** · **audit-strategy.md** · **data-classification.md** · **file-storage-strategy.md**
API y base de datos	**docs/architecture/api-conventions.md** · **database-strategy.md** · **initial-data-model.md**
Contexto y dominios	**docs/architecture/system-context.md** · **domain-map.md** · **frontend-compatibility.md**
Configuración de GitHub	**docs/development/github-configuration-report.md**
Publicación	**docs/development/release-process.md** (proceso formal pendiente)
Problemas frecuentes	**docs/development/troubleshooting.md**
Decisiones	**docs/decisions/README.md** · **pending-decisions.md**
Requisitos	**docs/requirements/requirements-catalogue.md** · **traceability-matrix.md** · **functional-scope.md** · **glossary.md** · **change-control.md**

8. Brechas y discrepancias consolidadas

Este capítulo reúne en un solo lugar lo que el sistema no hace todavía y las diferencias entre la documentación y el código detectadas al preparar este manual. Es la lista de trabajo para que un equipo técnico no tenga que inferirlas.

8.1 Capacidades no implementadas

Capacidad	Asociado
Trabajos en segundo plano, colas, workers, Redis, Celery	ADR-0009 · RNF-REN-003 (Deferred)
Programación automática de tareas y respaldos (no hay **crontab** versionado)	**operations-monitoring.md** · RNF-OPE-001
Rotación, retención y cifrado de respaldos; copia fuera del host	RNF-RES-001/002 · PD-002
Retención y purga de bitácora y documentos	PD-003 · RNF-LEG-001
Servidor de aplicación de producción, imagen estática del frontend y proxy TLS versionados	**secure-deployment.md**
Pipeline de entrega continua y proceso de release	**release-process.md**
Rulesets, protección de ramas y **CODEOWNERS** operativos	**github-configuration-report.md** · PD-011 (relacionado)
MFA, restablecimiento de contraseña por correo, límite de tasa en login	—
Endpoint de permisos para el cliente (el menú no oculta módulos)	ADR-0005
Pantallas de bitácora, roles, credenciales/QR, emisión oficial y monitoreo	Sección 4.8
Endpoint que entrega el token de descarga de documentos	Sección 3.8
Pruebas de extremo a extremo y de carga; métricas y alertas externas	PD-001 · PD-010
Diccionario de datos final y scripts finales de base de datos	**docs/manuals/05** y **06**
Inventario y programa de alimentos, agenda, comunicación completa, reportería avanzada	PD-009 · **functional-scope.md**

8.2 Discrepancias entre documentación y código

Documento	Discrepancia	Qué hacer
`authorization-model.md`	Lista unos 26 permisos y modela `PermissionGrant`; el código tiene 39 permisos y usa `Role.permissions` (M2M).	Actualizar el documento.
`initial-data-model.md`	Nombres y modelos distintos (sección 6.1).	Marcarlo como histórico o alinearlo.

Documento	Discrepancia	Qué hacer
`file-storage-strategy.md`	Cita `FileObject`, `DocumentVersion` y `DownloadToken`; el código tiene `DocumentRecord` y `DocumentDownloadToken`. Habla de catálogo de tipos y normalización de imágenes que no existen.	Alinear.
`database-migrations.md`	"Estado actual" lista 6 apps; hay 12 y 93 migraciones.	Actualizar.
`local-setup.md`	Dice SQLite como base local inicial y usa los puertos 8001 y 4173; Docker y Makefile usan PostgreSQL, 8000 y 5173.	Unificar.
`backend/.env.example`	Define `SESSION_COOKIE_SECURE` y `CSRF_COOKIE_SECURE`, que `base.py` ignora.	Eliminar o leerlas en settings.
`git-workflow.md`	No menciona Conventional Commits, que impone `pr-validation`.	Documentar.
`traceability-matrix.md`	Encabezado "todos Planned"; 25 requisitos sin fila; fila de RNF-SEG-005 malformada.	Corregir.
`pending-decisions.md`	No vincula PD a ADR ni a RF; PD-002 figura "Declarado" mientras RNF-RES-002 es *Planned*.	Añadir columnas de trazabilidad.
`backup-and-recovery.md`	Propone acotar la guardia de `pr-validation`; el workflow sigue con el patrón amplio (PD-011).	Decidir y aplicar.
`testing-strategy.md`	Habla de cobertura de frontend 60 %; el umbral de ramas es 50 %.	Precisar.
`api-conventions.md`	Sugiere `/api/v1/` "cuando exista implementación"; ya es el prefijo real.	Actualizar.
`Makefile`	`make check` invoca `build` (Docker), no el build del frontend; `DATABASE_PORT` fijo en 5432.	Renombrar o corregir.
`routes.jsx`	El comentario dice que el login no es perezoso; el código usa `React.lazy`.	Corregir el comentario.
Manual técnico anterior	Presentaba las dependencias como una jerarquía simple y el sobre de error sin matices; el capítulo 5 estaba vacío y faltaban los capítulos 6 y 7.	Sustituido por este documento.

Anexo A — Endpoints que consume el frontend

Rutas relativas a **/api/v1**. Se conservan del manual anterior tras revisarlas contra **frontend/src**. Los listados paginados aceptan **page**; los filtros en falso no se envían salvo **acknowledged=false**. La fuente exacta y completa de la API es **GET /api/v1/schema/**.

Endpoint	Uso	Filtros y parámetros
`GET /health/`	Salud del proceso, en el panel	Ninguno
`GET /auth/login/`	Dejar la cookie CSRF	Ninguno
`POST /auth/login/`	Abrir sesión	Cuerpo: username, password
`POST /auth/logout/`	Cerrar la sesión actual	Ninguno
`POST /auth/logout/all/`	Cerrar todas las sesiones	Ninguno
`GET /auth/me/`	Saber si hay sesión	Ninguno
`POST /auth/password/change/`	Cambiar la contraseña propia	Cuerpo: current_password, new_password
`GET /identity/accounts/list/`	Listar cuentas	page
`POST /identity/accounts/{id}/disable/`	Desactivar una cuenta	Cuerpo: force
`POST /identity/accounts/{id}/sessions/close/`	Cerrar sesiones de una cuenta	Ninguno
`POST /identity/accounts/{id}/password-resets/`	Emitir restablecimiento	Ninguno
`GET /people/`	Listar personas	page, search, include_inactive
`GET /people/{id}/`	Detalle de persona	Ninguno
`POST /people/`	Alta de persona	Cuerpo del formulario
`PATCH /people/{id}/`	Editar persona	Cuerpo del formulario
`DELETE /people/{id}/`	Baja lógica	Ninguno
`GET /students/`	Listar estudiantes	page
`GET /students/{id}/`	Detalle de expediente	Ninguno
`GET /students/next-code/`	Siguiente código libre	Ninguno
`POST /students/`	Alta de expediente	Cuerpo del formulario
`PATCH /students/{id}/`	Editar expediente o fotografía	Cuerpo o photo
`GET /students/{id}/health-notes/`	Notas de salud	Ninguno
`POST /students/{id}/health-notes/`	Nueva nota de salud	Cuerpo del formulario
`GET /students/{id}/observations/`	Observaciones	Ninguno
`POST /students/{id}/observations/`	Nueva observación	Cuerpo del formulario
`GET /students/{id}/emergency-contacts/`	Contactos de emergencia	Ninguno

Endpoint	Uso	Filtros y parámetros
`POST /students/{id}/emergency-contacts/`	Nuevo contacto	Cuerpo del formulario
`GET /students/guardian-relations/`	Vínculos estudiante-encargado	Ninguno
`POST /students/guardian-relations/`	Crear vínculo	Cuerpo del formulario
`GET /students/guardians/`	Listar encargados	page
`GET /students/guardians/{id}/`	Detalle de encargado	Ninguno
`POST /students/guardians/`	Alta de encargado	Cuerpo del formulario
`GET /teachers/`	Listar docentes	page
`GET /teachers/{id}/`	Detalle de docente	Ninguno
`GET /teachers/next-code/`	Siguiente código libre	Ninguno
`POST /teachers/`	Alta de docente	Cuerpo del formulario
`PATCH /teachers/{id}/`	Editar docente	Cuerpo o archivo
`GET /academics/cycles/`	Listar ciclos	page
`GET /academics/cycles/defaults/`	Fechas sugeridas de un año	year
`GET /academics/cycles/{id}/`	Detalle histórico del ciclo	Ninguno
`POST /academics/cycles/`	Crear ciclo	Cuerpo del formulario
`POST /academics/cycles/{id}/activate/`	Activar ciclo	Ninguno
`POST /academics/cycles/{id}/clone/`	Clonar ciclo	Cuerpo: incluye si se copian asignaciones
`GET /academics/campuses/`	Listar sedes	page, search, include_inactive
`GET /academics/campuses/next-code/`	Siguiente código de sede	Ninguno
`GET /academics/campuses/{id}/`	Detalle de sede	Ninguno
`POST /academics/campuses/`	Alta de sede	Cuerpo del formulario
`PATCH /academics/campuses/{id}/`	Editar sede	Cuerpo del formulario
`DELETE /academics/campuses/{id}/`	Baja lógica de sede	Ninguno
`GET /academics/classrooms/`	Listar aulas	page, include_inactive, campus_id
`GET /academics/classrooms/{id}/`	Detalle de aula	Ninguno
`POST /academics/classrooms/`	Alta de aula	Cuerpo del formulario
`PATCH /academics/classrooms/{id}/`	Editar aula	Cuerpo del formulario
`DELETE /academics/classrooms/{id}/`	Baja lógica de aula	Ninguno
`GET /academics/campuses/{id}/shifts/`	Jornadas de una sede	page

Endpoint	Uso	Filtros y parámetros
`POST /academics/campuses/{id}/shifts/`	Alta de jornada	Cuerpo del formulario
`GET /academics/shifts/{id}/`	Detalle de jornada	Ninguno
`PATCH /academics/shifts/{id}/`	Editar jornada	Cuerpo del formulario
`DELETE /academics/shifts/{id}/`	Baja lógica de jornada	Ninguno
`GET /academics/levels/`	Listar niveles	page, include_inactive
`GET /academics/levels/next-code/`	Siguiente código de nivel	Ninguno
`GET /academics/levels/{id}/`	Detalle de nivel	Ninguno
`POST /academics/levels/`	Alta de nivel	Cuerpo del formulario
`PATCH /academics/levels/{id}/`	Editar nivel	Cuerpo del formulario
`DELETE /academics/levels/{id}/`	Baja lógica de nivel	Ninguno
`GET /academics/levels/{id}/grades/`	Grados de un nivel	page, include_inactive
`POST /academics/levels/{id}/grades/`	Alta de grado	Cuerpo del formulario
`GET /academics/grades/{id}/`	Detalle de grado	Ninguno
`PATCH /academics/grades/{id}/`	Editar grado	Cuerpo del formulario
`DELETE /academics/grades/{id}/`	Baja lógica de grado	Ninguno
`GET /academics/subjects/`	Listar cursos	page, include_inactive
`GET /academics/subjects/{id}/`	Detalle de curso	Ninguno
`POST /academics/subjects/`	Alta de curso	Cuerpo del formulario
`PATCH /academics/subjects/{id}/`	Editar curso	Cuerpo del formulario
`DELETE /academics/subjects/{id}/`	Baja lógica de curso	Ninguno
`GET /academics/levels/{id}/subjects/`	Cursos de un nivel	page
`POST /academics/levels/{id}/subjects/`	Vincular curso	Cuerpo del formulario
`PATCH /academics/levels/{id}/subjects/{subjectId}/`	Editar vínculo	Cuerpo del formulario
`DELETE /academics/levels/{id}/subjects/{subjectId}/`	Quitar vínculo	Ninguno
`GET /academics/sections/`	Listar secciones	page
`GET /academics/teaching-assignments/history/`	Historial de asignaciones	page, academic_cycle_id
`POST /academics/teaching-assignments/`	Nueva asignación	Cuerpo: ciclo, sección, curso, docente
`POST /academics/teaching-assignments/{id}/reassignments/`	Reasignar	Cuerpo del formulario
`GET /enrolments/active/`	Matrículas vigentes	page, student_id
`GET /enrolments/history/`	Historial de un estudiante	page, student_id obligatorio

Endpoint	Uso	Filtros y parámetros
GET /enrolments/movements/	Movimientos	page, student_id
POST /enrolments/	Alta cruda de matrícula	Cuerpo del formulario
POST /enrolments/matriculations/	Matricular	Cuerpo del formulario
POST /enrolments/re-enrolments/	Reinscribir	Cuerpo del formulario
GET /enrolments/{id}/documents/	Documentos de la matrícula	page
POST /enrolments/{id}/documents/	Agregar documento	Cuerpo del formulario
GET /attendance/events/	Eventos de asistencia	page
POST /attendance/events/	Registrar evento manual	Cuerpo del formulario
GET /attendance/events/resolution/	Evento vigente tras duplicados	student_id, shift_id, event_date, movement_type
GET /attendance/day-status/	Estado del día	student_id, shift_id, event_date
GET /attendance/presence/	Quienes siguen dentro	shift_id obligatorio; event_date, grade_id, section_id
GET /attendance/attendance-percentage/	Porcentaje de un estudiante	student_id, shift_id
GET /attendance/jornada-parameters/	Parámetros de jornada	page
POST /attendance/jornada-parameters/	Crear parámetros	Cuerpo del formulario
POST /attendance/jornada-closures/	Cerrar jornada	Cuerpo: shift_id
GET /attendance/section-closures/preview/	Vista previa del cierre	section_id, event_date
POST /attendance/section-closures/	Cerrar sección	Cuerpo: section_id, event_date
GET /attendance/control-points/	Puntos de control	page
POST /attendance/scan/	Captura por escaneo	Cuerpo: items
GET /attendance/alerts/	Alertas de asistencia	page
GET /academics/evaluation-config/	Configuración global	Ninguno
PATCH /academics/evaluation-config/	Editar configuración global	Cuerpo del formulario
GET /academics/cycles/{id}/evaluation-config/	Configuración del ciclo	Ninguno
PATCH /academics/cycles/{id}/evaluation-config/	Editar configuración del ciclo	Cuerpo del formulario
GET /academics/cycles/{id}/evaluation-units/	Unidades del ciclo	page
POST /academics/cycles/{id}/evaluation-units/	Crear unidad	Cuerpo del formulario
PATCH /academics/cycles/{id}/evaluation-units/{unitId}/recovery-window/	Ventana de recuperación	Cuerpo del formulario
GET /academics/cycles/{id}/evaluation-units/{unitId}/capture-exceptions/	Excepciones de captura	page

Endpoint	Uso	Filtros y parámetros
POST /academics/cycles/{id}/evaluation-units/{unitId}/capture-exceptions/	Otorgar excepción	Cuerpo del formulario
GET /academics/cycles/{id}/evaluation-units/{unitId}/grades/	Notas de la unidad	page
POST /academics/cycles/{id}/evaluation-units/{unitId}/grades/	Registrar nota	Cuerpo del formulario
GET /academics/cycles/{id}/enrolments/{enrolmentId}/subjects/{subjectId}/current-average/	Promedio en curso	Ninguno
GET /academics/cycles/{id}/enrolments/{enrolmentId}/subjects/{subjectId}/final-grade/	Nota final	Ninguno
GET /documents/types/	Tipos de documento	page, include_inactive
POST /documents/types/	Alta de tipo	Cuerpo del formulario
PATCH /documents/types/{id}/	Editar tipo	Cuerpo del formulario
DELETE /documents/types/{id}/	Baja lógica del tipo	Ninguno
GET /documents/templates/	Plantillas	page, include_inactive
GET /documents/templates/{id}/	Detalle de plantilla	Ninguno
POST /documents/templates/	Alta de plantilla	Cuerpo del formulario
PATCH /documents/templates/{id}/	Editar plantilla	Cuerpo del formulario
DELETE /documents/templates/{id}/	Baja lógica de plantilla	Ninguno
GET /documents/templates/{id}/versions/	Versiones	page
POST /documents/templates/{id}/preview/	Vista previa	Cuerpo: payload
GET /documents/field-tags/	Etiquetas de plantilla	include_sensitive
GET /documents/enrolments/{id}/records/	Expediente documental	page
POST /documents/records/	Subir documento	Cuerpo del formulario
POST /documents/records/scan/	Escanear documento	Cuerpo del formulario
POST /documents/records/{id}/versions/	Nueva versión	Cuerpo del formulario
POST /documents/records/{id}/verify/	Verificar integridad	Ninguno
GET /documents/storage-consumption/	Consumo de almacenamiento	Ninguno
GET /documents/official-issuance/eligibility/	Si la matrícula puede emitir	enrolment_id
GET /reporting/alerts/	Alertas institucionales	page, alert_type, acknowledged
POST /reporting/alerts/{id}/acknowledge/	Marcar atendida	Ninguno

Endpoint	Uso	Filtros y parámetros
`POST /reporting/alert-evaluations/`	Recalcular alertas	Cuerpo: shift_id, event_date
`GET /reporting/absence-threshold-parameters/`	Umbrales de ausencia	page
`POST /reporting/absence-threshold-parameters/`	Crear umbral	Cuerpo: shift_id, academic_cycle_id

Anexo B — Inventario de tablas

Generado por introspección del registro de modelos de Django con **config.settings.test**: 69 modelos propios en 12 apps. Todos los modelos sa
public_id (UUID único), **created_at**, **updated_at** e **is_active**, omitidos de la columna de campos. “UQ parcial” es una restricción única con con
verificación. Este inventario es la base del diccionario de datos final (sección 6.4), que todavía debe completarse con tipos, tamaños y reglas d

App	Modelo (tabla)	Campos principales	Claves foráneas	Unicidad y restriccione
academics	**AcademicCycle** academics_academiccycle	year, name, description, starts_on, ends_on, status	institution → Institution	UQ (institution,name); (institution); CK academ academic_cycle_no_ov
academics	**Campus** academics_campus	name, code, address, is_main	institution → Institution	UQ (institution,code); U
academics	**ClassScheduleBlock** academics_classscheduleblock	number, name, starts_on, ends_on	shift → Shift	UQ (shift,number); CK c
academics	**ClassSchedulePublication** academics_classschedulepublication	published_at	academic_cycle → AcademicCycle (1:1)	-
academics	**ClassSession** academics_classsession	day_of_week, starts_on	academic_cycle → AcademicCycle; section → Section; subject → Subject; schedule_block → ClassScheduleBlock; classroom → Classroom	UQ (section,subject,day
academics	**Classroom** academics_classroom	name, code, location, capacity, service_status	campus → Campus	UQ (campus,code)
academics	**CurriculumPlan** academics_curriculumplan	is_required	academic_cycle → AcademicCycle; grade → Grade; subject → Subject	UQ (academic_cycle,gra
academics	**FrozenPromotionResult** academics_frozenpromotionresult	promoted, condition, failed_subjects, is_correction, correction_reason	enrolment → Enrolment	CK frozen_promotion_r
academics	**FrozenSubjectResult** academics_frozensubjectresult	final_grade, condition, recovery_grade, is_correction, correction_reason	enrolment → Enrolment; subject → Subject	CK frozen_subject_resu

App	Modelo (tabla)	Campos principales	Claves foráneas	Unicidad y restriccione
academics	**Grade** academics_grade	name, code, sequence	level → Level; institution → Institution	UQ (level,sequence); U
academics	**GradeOffering** academics_gradeoffering	—	academic_cycle → AcademicCycle; shift → Shift; grade → Grade	UQ (academic_cycle,shi
academics	**Institution** academics_institution	name, short_name	-	-
academics	**Level** academics_level	name, code, sequence	institution → Institution	UQ (institution,code); U (id,institution)
academics	**LevelSubject** academics_levelsubject	is_required, weekly_hours	level → Level; subject → Subject	UQ (level,subject)
academics	**Section** academics_section	name, capacity	offering → GradeOffering; default_classroom → Classroom	UQ (offering,name)
academics	**Shift** academics_shift	name, code	campus → Campus	UQ (campus,code)
academics	**Subject** academics_subject	name, code	institution → Institution; levels → M2M Level	UQ (institution,code)
academics	**TeachingAssignment** academics_teachingassignment	starts_on, ends_on	academic_cycle → AcademicCycle; section → Section; subject → Subject; teacher → Person	CK teaching_assignmen teaching_assignment_r (academic_cycle,sectio
attendance	**AttendanceAlert** attendance_attendancealert	alert_type, event_date, target_roles, context	student → Student; shift → Shift; section → Section	-
attendance	**AttendanceEvent** attendance_attendanceevent	event_date, movement_type, origin, transmission, captured_at, client_event_id, batch_id	student → Student; shift → Shift; control_point → ControlPoint; operator → UserAccount; manual_reason → ManualRegistrationReason; capture_batch → CaptureBatch	UQ parcial (client_even
attendance	**AttendancePermit** attendance_attendancepermit	permit_type, permit_date, scheduled_time, reason, status, resolved_at, resolution_comment	student → Student; submitted_by → UserAccount; resolved_by → UserAccount	-

App	Modelo (tabla)	Campos principales	Claves foráneas	Unicidad y restriccione
attendance	**CaptureBatch** attendance_capturebatch	status, confirmed_at, closed_at, session_key	operator → UserAccount	UQ parcial (operator)
attendance	**ControlPoint** attendance_controlpoint	name, code, allows_entry, allows_exit	campus → Campus	UQ (campus,code)
attendance	**JornadaParameters** attendance_jornadaparameters	entry_limit_time, tolerance_minutes, closing_time, duplicate_suppression_minutes, school_days, effective_from	shift → Shift; academic_cycle → AcademicCycle	UQ (shift,academic_cyc
attendance	**Justification** attendance_justification	absence_date, situation_type, reason, status, is_exception, exception_reason, resolved_at, resolution_comment	student → Student; reason_catalog → JustificationReason; submitted_by → UserAccount; resolved_by → UserAccount	-
attendance	**JustificationAttachment** attendance_justificationattachment	filename, storage_key, content_type, size_bytes, checksum	justification → Justification (1:1); uploaded_by → UserAccount	-
attendance	**JustificationNotification** attendance_justificationnotification	—	justification → Justification (1:1); recipient → UserAccount	-
attendance	**JustificationPolicy** attendance_justificationpolicy	window_business_days	-	-
attendance	**JustificationReason** attendance_justificationreason	name, code	-	-
attendance	**ManualRegistrationReason** attendance_manualregistrationreason	name, code	-	-
attendance	**SectionClosureLog** attendance_sectionclosurelog	event_date, is_covering, included_count, omitted_count	section → Section; declared_by → UserAccount	-
attendance	**StudentCredential** attendance_studentcredential	opaque_identifier, status, issued_at, revocation_reason	student → Student; revoked_by → UserAccount; revoked_on_movement → StudentMovement	UQ (opaque_identifier)

App	Modelo (tabla)	Campos principales	Claves foráneas	Unicidad y restriccione
audit	**AuditEvent** audit_auditevent	actor_label, action, resource, resource_identifier, ip_address, context	actor → UserAccount	-
common	**TaskRun** common_taskrun	name, status, started_at, finished_at, duration_ms, host, process_id, summary, error_type, error_message	-	-
documents	**DocumentBatchRun** documents_documentbatchrun	client_batch_id, document_type, enrolment_count	-	UQ parcial (client_batch
documents	**DocumentDeliveryReceipt** documents_documentdeliveryreceipt	document_type, folio, recipient_name, delivered_at, notes	student → Student; guardian → Guardian	-
documents	**DocumentDownloadToken** documents_documentdownloadtoken	token_hash, expires_at, used_at, revoked_at	document → DocumentRecord; created_by → UserAccount	-
documents	**DocumentKind** documents_documentkind	code, label, description	institution → Institution	UQ (institution,code)
documents	**DocumentRecord** documents_documentrecord	filename, storage_key, content_type, size_bytes, checksum, version_group_id, version_number, status	student → Student; enrolment → Enrolment; supersedes → DocumentRecord	UQ (version_group_id,v
documents	**DocumentTemplate** documents_documenttemplate	name, code, description, content	institution → Institution; document_kind → DocumentKind	UQ (institution,code); U (institution,document_
documents	**DocumentTemplateVersion** documents_documenttemplateversion	sequence, name, kind, description, content	template → DocumentTemplate	UQ (template,sequence
documents	**DocumentVerificationCode** documents_documentverificationcode	code, document_type, folio, issued_at	-	-
documents	**OfficialFolio** documents_officialfolio	year, sequence, document_type, issued_at	institution → Institution	UQ (institution,year,seq

App	Modelo (tabla)	Campos principales	Claves foráneas	Unicidad y restriccione
enrolments	**Enrolment** enrolments_enrolment	effective_on, ends_on, status	student → Student; academic_cycle → AcademicCycle; grade → Grade; section → Section	UQ parcial (student); U enrolment_valid_effect
enrolments	**EnrolmentDocumentRequirement** enrolments_enrolmentdocumentrequirement	code, name, is_required, status	enrolment → Enrolment	UQ (enrolment,code)
enrolments	**StudentMovement** enrolments_studentmovement	movement_type, effective_on, reason	student → Student; source_enrolment → Enrolment; target_enrolment → Enrolment	CK student_movement_ student_withdrawal_re student_movement_dis
enrolments	**StudentMovementAnnulment** enrolments_studentmovementannulment	reason	movement → StudentMovement (1:1); annulled_by → UserAccount	CK student_movement_
evaluation	**CaptureExceptionGrant** evaluation_captureexceptiongrant	reason, expires_at	evaluation_unit → EvaluationUnit; subject → Subject; teacher → Person	-
evaluation	**CycleEvaluationConfig** evaluation_cycleevaluationconfig	unit_count	academic_cycle → AcademicCycle (1:1)	CK cycle_evaluation_co
evaluation	**EvaluationGlobalConfig** evaluation_evaluationglobalconfig	singleton_key, default_unit_count	-	CK evaluation_global_c
evaluation	**EvaluationUnit** evaluation_evaluationunit	number, name, starts_on, ends_on, status, capture_starts_on, capture_ends_on, recovery_starts_on, recovery_ends_on	academic_cycle → AcademicCycle	UQ (academic_cycle,nu evaluation_unit_valid_c evaluation_unit_valid_c evaluation_unit_valid_r evaluation_unit_no_ov
evaluation	**Grade** evaluation_grade	value	enrolment → Enrolment; subject → Subject; evaluation_unit → EvaluationUnit	UQ (enrolment,subject, grade_value_within_sc
evaluation	**RecoveryGrade** evaluation_recoverygrade	value, original_final_grade	enrolment → Enrolment; subject → Subject	UQ (enrolment,subject) recovery_grade_value_

App	Modelo (tabla)	Campos principales	Claves foráneas	Unicidad y restriccione
identity	**ActivationChallenge** identity_activationchallenge	token_digest, expires_at, failed_attempts, used_at, revoked_at	account → UserAccount	-
identity	**PasswordResetChallenge** identity_passwordresetchallenge	token_digest, expires_at, used_at, revoked_at	account → UserAccount	-
identity	**Role** identity_role	name, slug, description, is_system, session_idle_timeout_minutes	permissions → M2M Permission	-
identity	**RoleAssignment** identity_roleassignment	starts_at, ends_at	user → UserAccount; role → Role	UQ (user,role,starts_at)
identity	**ScopeGrant** identity_scopegrant	module_key, starts_at, ends_at	assignment → RoleAssignment; institution → Institution; academic_cycle → AcademicCycle; grade → Grade; section → Section; subject → Subject; teaching_assignment → TeachingAssignment; student → Student	CK identity_scope_gran
identity	**UserAccount** identity_useraccount	password, last_login, is_superuser, username, first_name, last_name, email, is_staff, date_joined, status, failed_login_attempts, locked_until	person → Person (1:1); groups → M2M Group; user_permissions → M2M Permission	-
people	**Person** people_person	first_name, last_name, email, phone_number, institutional_identifier, birth_date	-	-
reporting	**AbsenceThresholdParameters** reporting_absencethresholdparameters	max_absences, lookback_days, effective_from	shift → Shift; academic_cycle → AcademicCycle	UQ (shift,academic_cyc
reporting	**Alert** reporting_alert	alert_type, event_date, target_roles, context, acknowledged_at	student → Student; shift → Shift; section → Section; source_attendance_alert → AttendanceAlert; acknowledged_by → UserAccount	UQ parcial (student,shif
students	**EmergencyContact** students_emergencycontact	name, phone_number, relationship_label	student → Student	-
students	**Guardian** students_guardian	—	person → Person (1:1)	-

App	Modelo (tabla)	Campos principales	Claves foráneas	Unicidad y restriccione
students	**Student** students_student	student_code, status, photo	person → Person (1:1)	-
students	**StudentGuardianRelation** students_studentguardianrelation	relationship_label, is_primary, starts_at, ends_at	student → Student; guardian → Guardian	UQ parcial (student)
students	**StudentHealthNote** students_studenthealthnote	content, recorded_on	student → Student; author → UserAccount	-
students	**StudentObservation** students_studentobservation	description, observed_on	student → Student; author → UserAccount	-
teachers	**Teacher** teachers_teacher	employee_code, specialty, position, appointment_date, photo	person → Person (1:1)	-